Conception Technique :
Chat Support Client Instantané B2B

Alexis Bertrand
Marine HUARD
BUT MMI 3

Conception Technique : 1
Chat Support Client Instantané B2B 1
1. Contexte et Objectif 3
Contexte 3
Objectifs 3
Utilisateurs 3
Spécificité B2B 3
2. Hypothèses et Contraintes 4
Hypothèses métier 4
Contraintes techniques 4
Contraintes réglementaires 4
Contraintes organisationnelles 4
3. Vue d'Ensemble de la Solution 5
Architecture haut niveau 5
Flux données simplifié 6
Principe architectural clé 6
4. Architecture Globale 7
Choix architecture : Monolithe modulaire avec WebSocket temps réel 7
Stack technologique détaillée 7
Déploiement infrastructure 8
5. Composants 10
6. Flux Fonctionnel 25
7. Modèle de Données 30
8. API et Interface 35
9. Règles Métier 54
Règles conversations 54
Règles messages 55
Règles utilisateurs 56
Règles métriques et qualité 56
10. Gestion des Performances 58
11. Sécurité 64
12. Observabilité 69
13. Déploiement 74
14. Risques 79
15. Questions Ouvertes 83
16. Annexes 87

1. Contexte et Objectif

Contexte

Une entreprise e-commerce B2B souhaite remplacer son formulaire de contact traditionnel par un système de chat en direct permettant aux clients professionnels authentifiés d'échanger instantanément avec des conseillers support pour des questions après-vente (suivi commandes, retours, assistance produits).

Objectifs

- Réduire le temps de réponse moyen de plusieurs heures à moins de 3 minutes pour les clients B2B
- Améliorer la satisfaction client professionnel grâce à une résolution immédiate des problèmes après-vente
- Permettre aux conseillers de gérer 3 à 4 conversations simultanées
- Assurer la persistance des conversations en cas de rafraîchissement de page ou déconnexion réseau
- Identifier clairement les clients professionnels avec leur historique commandes et les conseillers dans chaque échange
- Fournir une interface intuitive accessible depuis le navigateur sans installation
- Garantir la disponibilité du service pendant les heures ouvrées avec un SLA de 99% minimum
- Contextualiser les conversations avec l'historique client (commandes, entreprise, contrats)

Utilisateurs

- **Clients B2B authentifiés** : professionnels ayant un compte actif, accès réservé aux clients existants uniquement, pas d'accès anonyme
- **Conseillers support** : équipe de 3 à 8 agents spécialisés après-vente, authentifiés et formés au contexte B2B

Spécificité B2B

- Volume réduit et prévisible : 15-25 conversations simultanées maximum aux heures de pointe
- Qualité de service prioritaire sur volume massif
- Conversations plus longues et techniques que B2C avec une durée moyenne estimée à 10-15 minutes
- Nécessité d'accès aux données client structurées (commandes, contrats, historique interactions)

2. Hypothèses et Contraintes

Hypothèses métier

- Base clients limitée et identifiée : 200-500 comptes professionnels actifs
- Volume conversationnel prévisible : 30-80 conversations par jour ouvré
- Pic usage : 10h-12h et 14h-16h (horaires bureaux B2B)
- Taux conversion formulaire vers chat : 60-80% des demandes support migrent vers chat sous 3 mois
- Clients B2B équipés de navigateurs modernes : Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Connexion internet stable côté client : latence < 200ms acceptable
- Durée moyenne conversation : 10-15 minutes avec maximum 50 messages échangés
- Taux résolution premier contact : objectif 70% minimum

Contraintes techniques

- Budget infrastructure MVP limité : 50-100 euros par mois maximum
- Pas de serveur dédié disponible : utilisation obligatoire de services cloud managés
- Équipe développement réduite : 1-2 développeurs full-stack pendant 6 semaines
- Stack technique imposée : Node.js backend obligatoire pour cohérence avec infrastructure existante
- Pas de service tiers payant pour chat : développement sur mesure requis
- Performance réseau variable : clients B2B parfois derrière VPN entreprise ou proxy
- Support navigateurs : Chrome, Firefox, Safari, Edge versions récentes uniquement, pas de support IE11

Contraintes réglementaires

- Conformité RGPD obligatoire : données clients européens principalement
- Conservation conversations : 3 ans minimum pour obligation légale contrats B2B
- Traçabilité complète : audit trail de toutes modifications conversations
- Chiffrement données en transit et au repos : HTTPS/WSS obligatoire
- Droit à l'oubli : procédure suppression données client sur demande écrite

Contraintes organisationnelles

- Conseillers travaillent en horaires décalés : 8h-20h du lundi au vendredi
- Formation conseillers limitée : maximum 2 heures par agent
- Coexistence temporaire email et chat : migration progressive sur 6 mois
- Pas de refonte CRM existant : intégration légère uniquement
- Validation métier requise toutes les 2 semaines : démos sprint obligatoires

3. Vue d'Ensemble de la Solution

Architecture haut niveau

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          COUCHE PRÉSENTATION                             │
├──────────────────────────────────┬───────────────────────────────────────┤
│   Application Client B2B         │   Application Conseiller              │
│   (React + Socket.IO Client)     │   (React + Socket.IO Client)          │
│                                  │                                       │
│   - Interface chat               │   - Dashboard conversations           │
│   - Historique messages          │   - Panel contexte client             │
│   - Notifications temps réel     │   - Multi-conversations tabs          │
│   - Indicateurs typing           │   - Métriques temps réel              │
└──────────────────────────────────┴───────────────────────────────────────┘
                  │                                   │
                  │                                   │
                  │        HTTPS/WSS (Port 443)       │
                  │                                   │
                  └───────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                     COUCHE REVERSE PROXY (optionnel)                     │
│                          NGINX / Caddy                                   │
│   - Terminaison SSL/TLS                                                  │
│   - Load balancing Socket.IO (sticky sessions)                           │
│   - Rate limiting basique                                                │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                          COUCHE APPLICATION                              │
├──────────────────────────────────────────────────────────────────────────┤
│                     Serveur Node.js + Express                            │
│                                                                          │
│   ┌──────────────────────┐        ┌──────────────────────┐              │
│   │  API REST            │        │  Socket.IO Server    │              │
│   │  (Express routes)    │        │                      │              │
│   │                      │        │  - Connection handler│              │
│   │  - Auth endpoints    │        │  - Event listeners   │              │
│   │  - Conversations     │        │  - Room management   │              │
│   │  - Messages CRUD     │        │  - Broadcast logic   │              │
│   │  - User profile      │        │  - Redis adapter     │              │
│   │  - Métriques         │        │                      │              │
│   └──────────────────────┘        └──────────────────────┘              │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────┐      │
│   │                Middlewares Transverses                       │      │
│   │  - JWT verification (Supabase token)                         │      │
│   │  - Rate limiting (express-rate-limit)                        │      │
│   │  - Error handling global                                     │      │
│   │  - Request logging (Winston)                                 │      │
│   │  - CORS configuration                                        │      │
│   └──────────────────────────────────────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                  ┌───────────────┴───────────────┐
                  │               │               │
                  ▼               ▼               ▼
┌──────────────────────┐  ┌──────────────┐  ┌──────────────────────┐
│  COUCHE DONNÉES      │  │  CACHE       │  │  AUTHENTIFICATION    │
│                      │  │              │  │                      │
│  Supabase            │  │  Redis       │  │  Supabase Auth       │
│  (PostgreSQL)        │  │  (Upstash)   │  │                      │
│                      │  │              │  │  - JWT tokens        │
│  - Users             │  │  - Sessions  │  │  - User sessions     │
│  - Companies         │  │  - Typing    │  │  - Refresh tokens    │
│  - Conversations     │  │  - Presence  │  │  - Password reset    │
│  - Messages          │  │  - Cache     │  │                      │
│  - Orders            │  │              │  │                      │
│                      │  │              │  │                      │
│  Row Level Security  │  │  Durée TTL   │  │  Email verification  │
│  activé              │  │  < 5 min     │  │  optionnelle         │
└──────────────────────┘  └──────────────┘  └──────────────────────┘
```

Flux données simplifié

1. Client B2B se connecte via Néon Auth et reçoit un JWT token
2. Frontend établit connexion WebSocket Socket.IO en transmettant JWT dans handshake
3. Serveur valide JWT et associe socket à user ID puis place socket dans room personnelle
4. Client envoie message via Socket.IO event `message:send`
5. Serveur insère message dans PostgreSQL via Prisma ORM
6. Serveur broadcast message via Socket.IO aux sockets dans room conversation concernée
7. Destinataire reçoit message temps réel et marque comme lu automatiquement si conversation ouverte
8. Accusé lecture remonté serveur qui update PostgreSQL et broadcast event `message:read`
9. Redis cache stocke temporairement état typing indicators et présence utilisateurs connectés

Principe architectural clé

Architecture événementielle hybride combinant API REST synchrone pour opérations CRUD classiques et WebSocket bidirectionnel pour communications temps réel. Séparation stricte responsabilités : PostgreSQL source de vérité pour données persistantes, Redis cache éphémère pour état volatile, Socket.IO transport temps réel avec fallback automatique HTTP long-polling si WebSocket bloqué par proxy.

4. Architecture Globale

Choix architecture : Monolithe modulaire avec WebSocket temps réel

Justification choix monolithe plutôt que microservices :

- Volume limité B2B ne justifie pas complexité microservices
- Équipe réduite incapable maintenir multiples services
- Latence inter-services pénaliserait expérience temps réel
- Coût infrastructure multiplié avec orchestration Kubernetes
- Time-to-market MVP favorisé par simplicité déploiement

Le monolithe reste modulaire via découpage interne :

- Module Auth isolé réutilisable
- Module Chat découplé via events
- Module Metrics extraction facilitée si besoin futur

Stack technologique détaillée

Frontend :

- React 18.2 avec hooks et Context API
- TypeScript 5.3 pour typage statique fort
- Vite 5.0 comme bundler et dev server rapide
- Zustand 4.4 pour state management léger
- React Query 5.0 pour cache requêtes API
- Socket.IO Client 4.6 pour WebSocket
- Tailwind CSS 3.4 pour styling utilitaire
- React Hook Form 7.49 pour gestion formulaires performante
- Zod 3.22 pour validation schemas côté client
- date-fns 3.0 manipulation dates localisation française
- DOMPurify 3.0 sanitization HTML prévention XSS

Backend :

- Node.js 20 LTS version stable long terme
- Express.js 4.18 framework web minimaliste
- TypeScript 5.3 cohérence typage avec frontend
- Socket.IO 4.6 serveur WebSocket avec fallbacks
- Prisma 5.8 ORM type-safe avec migrations automatiques
- Néon JS SDK 2.39 client authentification
- Winston 3.11 logging structuré JSON
- Helmet 7.1 sécurisation headers HTTP
- express-rate-limit 7.1 protection DDoS basique
- bcrypt 5.1 hashage passwords si auth custom
- cors 2.8 configuration CORS stricte

Infrastructure :

- PostgreSQL 15 via Néon managed database
- Redis 7 via Upstash serverless pour cache optionnel
- NGINX 1.24 reverse proxy si hébergement VPS
- Docker 24 conteneurisation optionnelle développement

Justification Néon :

- Backend-as-a-Service évitant setup PostgreSQL manuel
- Authentification JWT prête à l'emploi avec refresh tokens
- Row Level Security PostgreSQL natif isolant données conversations
- Interface admin graphique facilitant debug
- Webhooks database disponibles si besoin événements
- Backup automatiques quotidiens inclus
- Coût MVP gratuit jusqu'à 500MB et 50k utilisateurs actifs mensuels
- Migration simple vers PostgreSQL self-hosted si nécessaire futur

Justification Socket.IO vs alternatives :

Comparaison WebSocket natif, Socket.IO, SSE, HTTP polling :
Socket.IO choisi car :

- Fallback automatique HTTP long-polling si WebSocket bloqué proxy
- Reconnexion automatique gérée nativement avec buffer messages
- Room management intégré simplifiant broadcast ciblé
- Support binaire natif pour futurs attachments
- Large communauté et maturité production prouvée
- Middleware auth simple via handshake query

WebSocket natif rejeté car nécessite implémentation manuelle reconnexion et rooms.
Server-Sent Events rejeté car unidirectionnel serveur vers client uniquement.

Déploiement infrastructure

Option 1 (recommandée MVP) : PaaS managé

- Frontend : Vercel ou Netlify déploiement automatique depuis GitHub
- Backend : Railway.app ou Render.com avec auto-scaling
- Database : Néon hosted PostgreSQL
- Cache : Upstash Redis serverless
- Monitoring : Sentry error tracking gratuit

Avantages :

- Déploiement simplifié sans configuration serveurs
- Scaling automatique si pic usage
- HTTPS/WSS certificates gérés automatiquement
- Coût mensuel 5-40 euros MVP
- Rollback instantané si déploiement cassé

Option 2 : VPS auto-géré

- Serveur VPS OVH/Hetzner 4 vCPU / 8GB RAM
- Docker Compose orchestrant backend + PostgreSQL + Redis + NGINX
- Certificates Let's Encrypt automatiques via Certbot
- Backup quotidiens automatisés rsync

Avantages :

- Coût fixe prévisible 15-30 euros/mois
- Contrôle total infrastructure

Inconvénients :

- Maintenance système requise (updates sécurité)
- Pas de scaling automatique
- Complexité déploiement accrue

Décision MVP : Option 1 PaaS pour time-to-market, migration Option 2 envisageable après validation pilote si coûts justifient.

Gestion environnements :

Trois environnements distincts :

- **Development** : local machine développeurs, base SQLite optionnelle
- **Staging** : réplique production pour tests QA avant déploiement
- **Production** : environnement client final

Variables environnement gérées via fichiers `.env` et secrets managés plateforme :

```
NODE_ENV=production
PORT=3001
DATABASE_URL=postgresql://user:pass@host:5432/chatdb
Néon_URL=https://xxx.Néon.co
Néon_ANON_KEY=eyJxxx...
Néon_SERVICE_KEY=eyJxxx...
REDIS_URL=redis://default:pass@host:6379
JWT_SECRET=random-256-bit-secret
CORS_ORIGIN=https://app.example.com
SENTRY_DSN=https://xxx@sentry.io/xxx
```

5. Composants

Composant 1 : Frontend Client B2B

Responsabilité : Interface utilisateur permettant aux clients professionnels authentifiés d'initier et participer à des conversations support.

Architecture interne :

- Pages principales : Login, Dashboard conversations, Vue conversation unique
- Composants réutilisables : MessageBubble, ConversationList, TypingIndicator, NotificationToast
- State management : Zustand stores pour user session, conversations actives, messages cache
- Hooks custom : useSocket (connexion Socket.IO), useMessages (pagination messages), useTyping

Interactions externes :

- Appels API REST backend pour chargement initial conversations et historique messages
- Connexion WebSocket Socket.IO persistante pour événements temps réel
- Envoi events Socket.IO : `message:send`, `typing:start`, `typing:stop`, `message:read`
- Réception events Socket.IO : `message:receive`, `conversation:update`, `typing`, `error`

Technologies spécifiques :

- React Router pour navigation SPA
- React Query cache requêtes API avec invalidation automatique
- Tailwind CSS avec design system personnalisé variables CSS
- LocalStorage pour persistence token JWT et préférences utilisateur

Composant 2 : Frontend Dashboard Conseiller

Responsabilité : Interface multi-conversations permettant aux conseillers de gérer plusieurs chats simultanés avec accès contexte client enrichi.

Différences vs Frontend Client :

- Layout multi-panels avec sidebar conversations en attente et actives
- Tabs conversations actives avec indicateurs messages non lus
- Panel latéral affichant historique commandes client sélectionné chargé via API REST
- Notifications sonores configurables via Web Audio API
- Dashboard métriques temps réel : nombre conversations actives, temps réponse moyen, satisfaction

Fonctionnalités spécifiques :

- Prise en charge conversation depuis liste d'attente via event `conversation:claim`
- Transfert conversation autre conseiller via modal sélection + event `conversation:transfer`
- Clôture conversation avec formulaire note résolution obligatoire
- Réponses pré-formatées via bibliothèque snippets chargée au login

Technologies additionnelles :

- Recharts pour graphiques métriques simples
- React DnD si drag-and-drop transfert conversations implémenté

Composant 3 : API REST Backend

Responsabilité : Exposition endpoints HTTP pour opérations CRUD conversations, messages, utilisateurs et récupération données métier (commandes).

Routes principales :

```
POST   /api/auth/login              - Authentification retournant JWT
POST   /api/auth/refresh            - Refresh access token
POST   /api/auth/logout             - Invalidation token

GET    /api/conversations           - Liste conversations utilisateur authentifié
POST   /api/conversations           - Création nouvelle conversation
GET    /api/conversations/:id       - Détails conversation spécifique
PATCH  /api/conversations/:id       - Update statut conversation
DELETE /api/conversations/:id       - Soft delete conversation

GET    /api/conversations/:id/messages  - Historique messages paginé
POST   /api/conversations/:id/messages  - Envoi message via REST (fallback)

GET    /api/users/me                - Profil utilisateur authentifié
GET    /api/users/:id/orders        - Historique commandes client (conseillers only)

GET    /api/metrics                 - Métriques dashboard conseiller
GET    /api/health                  - Health check endpoint monitoring
```

Middlewares appliqués :

- `authenticateJWT` : vérification token Néon chaque requête
- `authorize(['advisor'])` : restriction routes sensibles conseillers uniquement
- `rateLimitAPI` : 100 requêtes par 15 minutes par IP
- `validateRequest(schema)` : validation Zod payloads entrants
- `errorHandler` : catch erreurs globales retournant JSON structuré

Structure réponse standardisée :

```json
{
 "success": true,
 "data": { ... },
 "meta": {
   "timestamp": "2024-01-15T10:30:00Z",
   "pagination": {
     "page": 1,
     "perPage": 50,
     "total": 150
   }
 }
}
```

Gestion erreurs HTTP codes standards :

- 200 OK succès
- 201 Created ressource créée
- 400 Bad Request validation échouée
- 401 Unauthorized token manquant ou invalide
- 403 Forbidden droits insuffisants
- 404 Not Found ressource inexistante
- 429 Too Many Requests rate limit dépassé
- 500 Internal Server Error erreur serveur non gérée

Composant 4 : Serveur Socket.IO
Responsabilité : Gestion connexions WebSocket persistantes et broadcast événements temps réel entre clients et conseillers.
Architecture événementielle :
Événements émis par clients :

- `message:send` : envoi nouveau message texte
- `typing:start` : utilisateur commence taper
- `typing:stop` : utilisateur arrête taper
- `message:read` : utilisateur a lu messages conversation
- `conversation:claim` : conseiller prend en charge conversation
- `conversation:transfer` : conseiller transfère conversation
- `conversation:close` : conseiller clôture conversation

Événements émis par serveur :

- `message:receive` : nouveau message reçu conversation
- `typing` : notification typing indicator
- `conversation:update` : changement statut ou assignation conversation
- `notification` : alerte générique (nouvelle conversation en attente)
- `error` : erreur métier ou validation
- `connection_ack` : confirmation connexion réussie avec user data

Logique rooms Socket.IO :
Chaque socket rejoint automatiquement rooms suivantes après authentification :

- `user:{userId}` : room personnelle pour notifications individuelles
- `conversation:{conversationId}` : room par conversation active utilisateur
- `advisors-pool` : room globale tous conseillers connectés (notifications nouvelles conversations)

Exemple implémentation handler :

```
io.on('connection', async (socket) => {
 const userId = socket.data.userId; // Extrait depuis JWT handshake
 const user = await prisma.user.findUnique({ where: { id: userId } });

 // Joindre rooms pertinentes
 socket.join(`user:${userId}`);
  if (user.role === 'ADVISOR') {
   socket.join('advisors-pool');
 }

 // Charger conversations actives utilisateur
 const conversations = await prisma.conversation.findMany({
```

```
  where: {
    OR: [
      { clientId: userId },
      { advisorId: userId }
    ],
    status: { in: ['WAITING', 'ACTIVE'] }
  }
});

conversations.forEach(conv => {
  socket.join(`conversation:${conv.id}`);
});

// Handler envoi message
socket.on('message:send', async (payload) => {
  const { conversationId, content, tempId } = payload;

  // Validation
  if (!content || content.length > 5000) {
    return socket.emit('error', {
      code: 'INVALID_MESSAGE',
      message: 'Message vide ou trop long',
      tempId
    });
  }

  // Vérification droits conversation
  const conversation = await prisma.conversation.findFirst({
    where: {
      id: conversationId,
      OR: [
        { clientId: userId },
        { advisorId: userId }
```

```
      ]
    }
  });

  if (!conversation) {
    return socket.emit('error', {
      code: 'CONVERSATION_NOT_FOUND',
      tempId
    });
  }

  // Insertion message database
  const message = await prisma.message.create({
    data: {
      conversationId,
      senderId: userId,
      content,
      type: 'TEXT',
      sentAt: new Date()
    },
    include: {
      sender: {
        select: { id: true, name: true, role: true }
      }
    }
  });

  // Broadcast room conversation
  io.to(`conversation:${conversationId}`).emit('message:receive', {
    conversationId,
    message,
    tempId
  });
```

```
  // Update timestamp dernière activité conversation
  await prisma.conversation.update({
    where: { id: conversationId },
    data: { updatedAt: new Date() }
  });
});

// Handler typing indicators
socket.on('typing:start', ({ conversationId }) => {
  socket.to(`conversation:${conversationId}`).emit('typing', {
    conversationId,
    userId,
    userName: user.name,
    isTyping: true
  });
});

socket.on('typing:stop', ({ conversationId }) => {
  socket.to(`conversation:${conversationId}`).emit('typing', {
    conversationId,
    userId,
    isTyping: false
  });
});

socket.on('disconnect', () => {
  console.log(`User ${userId} disconnected`);
  // Cleanup automatique rooms par Socket.IO
});
});
```

Gestion reconnexion :
Socket.IO gère automatiquement reconnexion avec backoff exponentiel (1s, 2s, 4s, 8s jusqu'à max 60s).
Côté client implémentation custom hook :

```
const useSocket = () => {
 const socket = useRef<Socket | null>(null);
 const [isConnected, setIsConnected] = useState(false);

 useEffect(() => {
   const token = localStorage.getItem('access_token');

   socket.current = io('wss://api.example.com', {
     auth: { token },
     transports: ['websocket', 'polling'],
     reconnection: true,
     reconnectionAttempts: 10,
     reconnectionDelay: 1000,
     reconnectionDelayMax: 60000
   });

   socket.current.on('connect', () => {
     setIsConnected(true);
     console.log('Socket connected:', socket.current.id);
   });

   socket.current.on('disconnect', (reason) => {
     setIsConnected(false);
     console.log('Socket disconnected:', reason);
   });

   socket.current.on('connect_error', (error) => {
     console.error('Socket connection error:', error.message);
```

```
  });

  return () => {
    socket.current?.disconnect();
  };
}, []);

return { socket: socket.current, isConnected };
};
```

Composant 5 : Couche Persistence (Néon PostgreSQL + Prisma)
Responsabilité : Stockage durable données structurées conversations, messages, utilisateurs avec garanties ACID.
Choix Prisma ORM justifications :

- Type-safety complet TypeScript avec auto-complétion IDE
- Migrations versionnées automatiques via `prisma migrate`
- Relations modèles définies déclarativement dans schéma
- Query builder intuitif évitant SQL brut vulnérable injections
- Support transactions avec `prisma.$transaction()`
- Introspection base existante possible si migration legacy

Configuration Prisma :

```
generator client {
 provider = "prisma-client-js"
}

datasource db {
 provider = "postgresql"
 url      = env("DATABASE_URL")
}

model User {
 id        String   @id @default(uuid())
 email     String   @unique
 name      String
 role      Role     @default(CLIENT)
```

```
 companyId String?  @map("company_id")
 createdAt DateTime @default(now()) @map("created_at")
 updatedAt DateTime @updatedAt @map("updated_at")

 company              Company?       @relation(fields: [companyId],
references: [id], onDelete: SetNull)
 sentMessages         Message[]
 clientConversations  Conversation[] @relation("ClientConversations")
 advisorConversations Conversation[] @relation("AdvisorConversations")

 @@index([email])
 @@index([companyId])
 @@map("users")
}

enum Role {
 CLIENT
 ADVISOR
 ADMIN
}

model Company {
 id        String   @id @default(uuid())
 name      String
 siret     String   @unique
 createdAt DateTime @default(now()) @map("created_at")

 users  User[]
 orders Order[]

 @@index([siret])
 @@map("companies")
}
```

```
model Conversation {
 id         String             @id @default(uuid())
 clientId   String             @map("client_id")
 advisorId  String?            @map("advisor_id")
 subject    String?            @db.VarChar(200)
 status     ConversationStatus @default(WAITING)
 createdAt  DateTime           @default(now()) @map("created_at")
 updatedAt  DateTime           @updatedAt @map("updated_at")
 closedAt   DateTime?          @map("closed_at")
 rating     Int?               @db.SmallInt
 metadata   Json               @default("{}")

 client   User      @relation("ClientConversations", fields: [clientId],
references: [id], onDelete: Cascade)
 advisor  User?     @relation("AdvisorConversations", fields: [advisorId],
references: [id], onDelete: SetNull)
 messages Message[]

 @@index([clientId, status])
 @@index([advisorId, status])
 @@index([status, createdAt(sort: Desc)])
 @@map("conversations")
}

enum ConversationStatus {
 WAITING
 ACTIVE
 RESOLVED
 CLOSED
}

model Message {
```

```
 id             String      @id @default(uuid())
 conversationId String      @map("conversation_id")
 senderId       String      @map("sender_id")
 content        String      @db.Text
 type           MessageType @default(TEXT)
 sentAt         DateTime    @default(now()) @map("sent_at")
 readAt         DateTime?   @map("read_at")
 metadata       Json        @default("{}")

 conversation Conversation @relation(fields: [conversationId], references:
[id], onDelete: Cascade)
 sender       User         @relation(fields: [senderId], references: [id],
onDelete: SetNull)

 @@index([conversationId, sentAt(sort: Desc)])
 @@index([senderId])
 @@map("messages")
}

enum MessageType {
 TEXT
 SYSTEM
}

model Order {
 id           String    @id @default(uuid())
 companyId    String    @map("company_id")
 orderNumber  String    @unique @map("order_number")
 amount       Decimal   @db.Decimal(10, 2)
 status       String
 deliveryDate DateTime? @map("delivery_date") @db.Date
 createdAt    DateTime  @default(now()) @map("created_at")
```

```
 company Company @relation(fields: [companyId], references: [id], onDelete:
Cascade)

 @@index([companyId, createdAt(sort: Desc)])
 @@map("orders")
}
```

Row Level Security PostgreSQL activé côté Néon :
Policies garantissant isolation données :

- Utilisateurs CLIENT voient uniquement leurs propres conversations
- Utilisateurs ADVISOR voient conversations assignées ou en attente
- Lecture messages limitée conversations accessibles utilisateur
- Modification statut conversation réservée conseillers assignés

Exemple policy SQL :

```sql
CREATE POLICY "Users can view their own conversations"
ON conversations FOR SELECT
USING (
 auth.uid() = client_id
 OR auth.uid() = advisor_id
 OR (status = 'WAITING' AND EXISTS (
   SELECT 1 FROM users WHERE id = auth.uid() AND role = 'ADVISOR'
 ))
);

CREATE POLICY "Advisors can update assigned conversations"
ON conversations FOR UPDATE
USING (
 EXISTS (
   SELECT 1 FROM users
   WHERE id = auth.uid()
   AND role IN ('ADVISOR', 'ADMIN')
 )
 AND (advisor_id = auth.uid() OR status = 'WAITING')
);
```

Composant 6 : Cache Redis (Upstash)
Responsabilité : Stockage temporaire données volatiles haute fréquence lecture/écriture pour décharger PostgreSQL.
Cas usage Redis :

- Sessions Socket.IO distribuées si multi-instances backend
- Typing indicators avec TTL 5 secondes
- Présence utilisateurs connectés avec heartbeat
- Rate limiting compteurs par IP/utilisateur
- Cache requêtes API métadonnées client (commandes récentes)

Structures données utilisées :

```
# Typing indicators
Key: typing:{conversationId}:{userId}
Type: String
Value: timestamp last typing event
TTL: 5 secondes

# Presence users
Key: presence:{userId}
Type: String
Value: JSON { socketId, lastSeen, status }
TTL: 30 secondes (renouvelé heartbeat)

# Rate limiting
Key: ratelimit:message:{userId}
Type: String
Value: compteur messages
TTL: 60 secondes (fenêtre glissante)

# Cache orders
Key: cache:orders:{companyId}
Type: String
Value: JSON array dernières 5 commandes
TTL: 300 secondes (5 minutes)

Client Redis configuration :
```

```
import { Redis } from '@upstash/redis';

const redis = new Redis({
 url: process.env.REDIS_URL,
 token: process.env.REDIS_TOKEN
});

// Helper typing indicator
export async function setTyping(conversationId: string, userId: string) {
 await redis.setex(
   `typing:${conversationId}:${userId}`,
   5,
   Date.now().toString()
 );
}

export async function getTypingUsers(conversationId: string):
Promise<string[]> {
 const keys = await redis.keys(`typing:${conversationId}:*`);
 return keys.map(key => key.split(':')[2]);
}

// Helper rate limiting
export async function checkRateLimit(userId: string, limit: number):
Promise<boolean> {
 const key = `ratelimit:message:${userId}`;
 const current = await redis.incr(key);
  if (current === 1) {
   await redis.expire(key, 60);
 }
  return current <= limit;
}
```

Fallback si Redis indisponible : application fonctionne en mode dégradé sans cache, typing indicators désactivés, rate limiting basic en mémoire Node.js.

6. Flux Fonctionnel

Flux 1 : Authentification Client

1. Client accède page login (https://app.example.com/login)
2. Saisie email + password dans formulaire React Hook Form
3. Validation Zod côté client (email format, password min 8 caractères)
4. Appel API POST /api/auth/login avec credentials
5. Backend appelle Néon Auth signInWithPassword()
6. Néon vérifie credentials dans table auth.users
7. Si valide : Néon génère access_token JWT (expire 1h) + refresh_token (expire 30j)
8. Backend retourne tokens + user profile (id, email, name, role)
9. Frontend stocke access_token localStorage + refresh_token httpOnly cookie
10. Redirection automatique dashboard conversations (/conversations)
11. Établissement connexion Socket.IO avec access_token dans auth header
12. Serveur Socket.IO valide JWT via Néon verifiant signature
13. Si valide : socket placé room user:{userId}
14. Frontend affiche interface chat avec état "connecté"

Gestion erreurs authentification :

- Credentials invalides : affichage message "Email ou mot de passe incorrect"
- Compte bloqué : message "Compte temporairement désactivé, contacter support"
- Token expiré : refresh automatique transparent via interceptor Axios
- Refresh token expiré : déconnexion forcée redirection login

Flux 2 : Création Conversation par Client

1. Client clique bouton "Nouvelle conversation" dashboard
2. Modal ouverture avec formulaire :
   - Subject (select dropdown) : Suivi commande, Retour produit, Question technique, Autre
   - Message initial (textarea obligatoire, max 1000 caractères)
3. Validation Zod : message non vide, subject sélectionné
4. Appel API POST /api/conversations avec payload :

```
{
  subject: "Suivi commande",
  initialMessage: "Bonjour, je souhaite connaître..."
}
```

5. Backend transaction Prisma :
   a. Création conversation status=WAITING, clientId=userId authentifié
   b. Insertion message initial senderId=clientId, type=TEXT
   c. Commit transaction

6. Backend émet Socket.IO broadcast room "advisors-pool" :
   Event: notification
   Payload: {
     type: "NEW_CONVERSATION",
     conversation: { id, subject, clientName, createdAt }
   }
7. API retourne conversation créée avec conversationId
8. Frontend rejoint room Socket.IO conversation:{conversationId}
9. Redirection automatique page conversation /conversations/{conversationId}
10. Affichage interface chat avec message initial + status "En attente de prise en charge"
11. Tous conseillers connectés reçoivent notification sonore + badge dashboard

Cas particuliers :

- Client a déjà 3 conversations WAITING : bloquer création avec message "Vous avez déjà 3 conversations en attente"
- Aucun conseiller connecté : conversation créée quand même, email envoyé équipe support

Flux 3 : Prise en Charge Conversation par Conseiller

1. Conseiller voit nouvelle conversation apparaître liste "En attente" dashboard temps réel
2. Conseiller clique bouton "Prendre en charge" conversation souhaitée
3. Frontend émet Socket.IO event :
   conversation:claim { conversationId }
4. Serveur reçoit event, lance transaction Prisma :
   a. Lecture conversation avec lock FOR UPDATE
   b. Vérification status=WAITING (pas déjà claimed par autre conseiller)
   c. Si WAITING : update advisorId=conseillerUserId, status=ACTIVE, claimedAt=now()
   d. Insertion message système type=SYSTEM : "[Conseiller Jean] a pris en charge la conversation"
   e. Commit transaction
5. Si transaction réussie :
   a. Serveur émet conversation:update room conversation:{conversationId}
      Payload: { conversationId, status: "ACTIVE", advisor: { id, name } }
   b. Serveur retire conversation de liste conseillers non assignés
   c. Client reçoit update affichant "[Conseiller Jean] vous répond"
6. Si transaction échoue (race condition autre conseiller plus rapide) :
   a. Serveur émet error au conseiller socket
   b. Frontend affiche toast "Conversation déjà prise en charge"
   c. Conversation retirée liste attente automatiquement
7. Conseiller voit conversation passer onglet "Actives" avec contexte client chargé
8. Panel latéral affiche historique 5 dernières commandes client via GET /api/users/{clientId}/orders

Règles métier prise en charge :

- Conseiller peut prendre max 4 conversations actives simultanées
- Vérification statut conseiller "disponible" (pas en pause)
- Priorité chronologique conversations en attente

Flux 4 : Envoi et Réception Message Temps Réel

1. Utilisateur tape message dans textarea interface chat
2. Debounce 300ms détecte frappe clavier
3. Frontend émet Socket.IO event typing:start { conversationId }
4. Serveur broadcast typing room conversation:{conversationId} (sauf émetteur)
5. Autres participants reçoivent typing indicator "Conseiller écrit..."
6. Si pause frappe > 3 secondes : frontend émet typing:stop
7. Utilisateur clique bouton "Envoyer" ou presse Ctrl+Enter
8. Frontend génère tempId unique (UUID) pour optimistic update
9. Message ajouté immédiatement interface avec status "Envoi..." (optimistic)
10. Frontend émet Socket.IO event :
    message:send {
      conversationId,
      content: "Votre commande a été expédiée ce matin",
      tempId
    }
11. Serveur reçoit event, validations :
    - Content non vide et < 5000 caractères
    - Utilisateur membre conversation (clientId ou advisorId)
    - Rate limit pas dépassé (20 messages/minute)
12. Si validation OK :
    a. Insertion message PostgreSQL via Prisma
    b. Broadcast Socket.IO room conversation:{conversationId} :
       Event: message:receive
       Payload: {
         message: { id, conversationId, senderId, content, sentAt, sender: {...} },
         tempId
       }
13. Frontend reçoit message:receive :
    a. Remplace message optimistic (tempId matchant) par message persisté (id database)
    b. Affiche message avec status "Envoyé" (checkmark simple)
14. Si destinataire conversation ouverte (tab focus) :
    a. Frontend émet immédiatement message:read { conversationId, messageIds: [...] }
    b. Serveur update readAt timestamp messages
    c. Broadcast message:read émetteur initial
    d. Émetteur voit checkmark double bleu "Lu"
15. Si destinataire hors ligne ou conversation fermée :
    - Message reste status "Envoyé" checkmark gris
    - Notification push envoyée si activée (browser notification API)

Gestion erreurs envoi :

- Message vide : bouton envoi disabled
- Rate limit dépassé : error toast "Trop de messages envoyés, patientez 1 minute"
- Erreur réseau : message reste status "Erreur envoi" avec bouton retry
- Validation serveur échouée : rollback optimistic update, affichage erreur

Flux 5 : Clôture Conversation et Rating Satisfaction

1. Conseiller estime problème résolu, clique "Marquer comme résolue"
2. Modal ouverture avec formulaire obligatoire :
   - Catégorie résolution (select) : Suivi commande, Retour, Facturation, Technique, Autre
   - Note interne (textarea 50-500 caractères) : "Commande réexpédiée, numéro suivi fourni"
3. Validation Zod côté client
4. Frontend appel API PATCH /api/conversations/{id} :
```
{
  status: "RESOLVED",
  resolutionCategory: "Suivi commande",
  resolutionNote: "Commande réexpédiée..."
}
```
5. Backend transaction Prisma :
   a. Update conversation status=RESOLVED, closedAt=now()
   b. Stockage metadata JSON résolution
   c. Insertion message système "Conversation marquée résolue par [Conseiller]"
   d. Commit transaction
6. Backend broadcast Socket.IO conversation:update room conversation:{conversationId}
7. Client reçoit update, interface affiche :
   - Banner "Cette conversation a été résolue"
   - Message système dans timeline
   - Modal invitation rating satisfaction
8. Modal rating client :
   - Étoiles 1-5 cliquables
   - Textarea commentaire optionnel
   - Bouton "Envoyer feedback" ou "Passer"
9. Si client note :
   a. Appel API PATCH /api/conversations/{id}/rating
      { rating: 5, feedback: "Merci pour votre réactivité" }
   b. Backend update conversation rating + metadata.feedback
   c. Conversation passe status=CLOSED
10. Si client ignore modal ou timeout 48h :
    - Cron job backend détecte conversations RESOLVED > 48h
    - Transition automatique status=CLOSED sans rating
11. Conversation archivée déplacée onglet "Historique" interface client et conseiller
12. Possibilité réouvrir conversation < 7 jours :
    - Bouton "Rouvrir cette conversation" visible historique
    - Crée nouvelle conversation liée ancienne référence metadata

Règles métier clôture :

- Seul conseiller assigné peut clôturer conversation
- Clôture impossible si conversation pas status ACTIVE
- Note résolution obligatoire traçabilité qualité
- Rating optionnel mais fortement encouragé UX

Flux 6 : Reconnexion Automatique après Déconnexion Réseau

1. Utilisateur perd connexion réseau (WiFi coupé, mobile 4G faible)
2. Socket.IO détecte disconnect event côté client après heartbeat timeout (10s)
3. Frontend affiche banner "Connexion perdue, reconnexion en cours..."
4. Socket.IO client tente reconnexion automatique avec backoff exponentiel :
   - Tentative 1 après 1 seconde
   - Tentative 2 après 2 secondes
   - Tentative 3 après 4 secondes
   - ... jusqu'à max 60 secondes entre tentatives
5. Pendant reconnexion :
   - Messages saisis interface stockés queue locale (IndexedDB)
   - Interface reste responsive en lecture seule
   - Bouton envoi disabled avec tooltip "Reconnexion..."
6. Réseau revient, connexion WebSocket rétablie :
   a. Socket.IO émet connect event
   b. Handshake authentification re-effectué avec access_token
   c. Serveur valide JWT toujours valide (pas expiré)
   d. Socket rejoint automatiquement rooms conversations actives
7. Frontend synchronisation delta messages manqués :
   a. Appel API GET /api/conversations/{id}/messages?since={lastMessageId}
   b. Backend retourne messages créés depuis déconnexion
   c. Frontend insert messages manqués timeline chronologique
8. Messages queue locale envoyés séquentiellement :
   - Pour chaque message queue : émet message:send avec tempId original
   - Attend accusé réception serveur avant envoyer suivant
9. Banner "Connecté" affichée 3 secondes puis disparaît
10. Interface retourne état normal fonctionnel

Cas limites reconnexion :

- Access token expiré pendant déconnexion : refresh automatique silent via refresh_token avant reconnexion
- Refresh token expiré : déconnexion forcée, redirect login avec message "Session expirée"
- Conversation clôturée pendant déconnexion : affichage info "Conversation fermée pendant votre absence"

7. Modèle de Données

Schéma relationnel complet avec contraintes :

```
┌──────────────────────────────────────────────────────┐
│                    users                             │
├──────────────────────────────────────────────────────┤
│ id            UUID PK                                │
│ email         VARCHAR(255) UNIQUE NOT NULL           │
│ name          VARCHAR(100) NOT NULL                  │
│ role          ENUM NOT NULL                          │
│               (CLIENT, ADVISOR, ADMIN)               │
│ company_id    UUID FK → companies.id                 │
│ created_at    TIMESTAMP DEFAULT NOW()                │
│ updated_at    TIMESTAMP DEFAULT NOW()                │
├──────────────────────────────────────────────────────┤
│ INDEX idx_users_email (email)                        │
│ INDEX idx_users_company (company_id)                 │
└──────────────────────────────────────────────────────┘
        │
        │ 1:N
        ▼

┌──────────────────────────────────────────────────────┐
│                  companies                           │
├──────────────────────────────────────────────────────┤
│ id            UUID PK                                │
│ name          VARCHAR(200) NOT NULL                  │
│ siret         VARCHAR(14) UNIQUE NOT NULL            │
│ created_at    TIMESTAMP DEFAULT NOW()                │
├──────────────────────────────────────────────────────┤
│ INDEX idx_companies_siret (siret)                    │
└──────────────────────────────────────────────────────┘
        │
        │ 1:N
        ▼
```

```
┌──────────────────────────────────────────────────────────┐
│                    orders                                │
├──────────────────────────────────────────────────────────┤
│ id              UUID PK                                  │
│ company_id      UUID FK NOT NULL                         │
│ order_number    VARCHAR(50) UNIQUE NOT NULL              │
│ amount          DECIMAL(10,2) NOT NULL                   │
│ status          VARCHAR(50) NOT NULL                     │
│ delivery_date   DATE                                     │
│ created_at      TIMESTAMP DEFAULT NOW()                  │
├──────────────────────────────────────────────────────────┤
│ INDEX idx_orders_company (company_id,                    │
│                           created_at DESC)               │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────┐
│                  conversations                           │
├──────────────────────────────────────────────────────────┤
│ id              UUID PK                                  │
│ client_id       UUID FK NOT NULL → users.id              │
│ advisor_id      UUID FK → users.id                       │
│ subject         VARCHAR(200)                             │
│ status          ENUM NOT NULL                            │
│                 (WAITING, ACTIVE,                        │
│                  RESOLVED, CLOSED)                       │
│ created_at      TIMESTAMP DEFAULT NOW()                  │
│ updated_at      TIMESTAMP DEFAULT NOW()                  │
│ closed_at       TIMESTAMP                                │
│ rating          SMALLINT CHECK (1-5)                     │
│ metadata        JSONB DEFAULT '{}'                       │
│                 - resolutionCategory                     │
│                 - resolutionNote                         │
│                 - feedback                               │
│                 - transferHistory []                     │
├──────────────────────────────────────────────────────────┤
│ INDEX idx_conv_client_status                             │
│       (client_id, status)                                │
│ INDEX idx_conv_advisor_status                            │
│       (advisor_id, status)                               │
│ INDEX idx_conv_status_created                            │
│       (status, created_at DESC)                          │
└──────────────────────────────────────────────────────────┘
```

```
        │
        │ 1:N
        ▼
┌──────────────────────────────────────────────────────────┐
│                    messages                              │
├──────────────────────────────────────────────────────────┤
│ id              UUID PK                                  │
│ conversation_id UUID FK NOT NULL                         │
│ sender_id       UUID FK NOT NULL → users.id              │
│ content         TEXT NOT NULL                            │
│ type            ENUM DEFAULT TEXT                        │
│                 (TEXT, SYSTEM)                           │
│ sent_at         TIMESTAMP DEFAULT NOW()                  │
│ read_at         TIMESTAMP                                │
│ metadata        JSONB DEFAULT '{}'                       │
├──────────────────────────────────────────────────────────┤
│ INDEX idx_messages_conv_sent                             │
│       (conversation_id, sent_at DESC)                    │
│ INDEX idx_messages_sender (sender_id)                    │
└──────────────────────────────────────────────────────────┘
```

Règles intégrité référentielle :

- `users.company_id → companies.id` : ON DELETE SET NULL (si entreprise supprimée, utilisateurs conservés orphelins)
- `conversations.client_id → users.id` : ON DELETE CASCADE (suppression client entraîne suppression conversations)
- `conversations.advisor_id → users.id` : ON DELETE SET NULL (si conseiller supprimé, conversations conservées sans assignation)
- `messages.conversation_id → conversations.id` : ON DELETE CASCADE (suppression conversation supprime messages associés)
- `messages.sender_id → users.id` : ON DELETE SET NULL (messages conservés même si utilisateur supprimé, sender_id devient NULL)
- `orders.company_id → companies.id` : ON DELETE CASCADE

Volumétrie estimée année 1 :

- Users : 500 clients + 10 conseillers = 510 rows
- Companies : 300 entreprises = 300 rows
- Conversations : 80 conv/jour × 250 jours ouvrés = 20 000 rows
- Messages : moyenne 30 messages/conv × 20 000 = 600 000 rows
- Orders : non géré par chat, base existante ~50 000 rows

Taille stockage estimée :

- Messages (600k rows × 500 bytes avg) = 300 MB
- Conversations (20k rows × 300 bytes) = 6 MB
- Users + Companies + Orders = 50 MB
- Total année 1 : **~350-400 MB** largement sous limite gratuite Néon 500MB

Stratégie partitionnement (anticipation croissance) :

Si volumétrie dépasse 5 millions messages (horizon 5-7 ans), mise en place partitionnement PostgreSQL par date :

```
CREATE TABLE messages (
  -- colonnes existantes
) PARTITION BY RANGE (sent_at);

CREATE TABLE messages_2024_q1 PARTITION OF messages
  FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');

CREATE TABLE messages_2024_q2 PARTITION OF messages
  FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

Permet archivage partitions anciennes vers stockage froid après 3 ans rétention légale.

Champs metadata JSON détails :

`conversations.metadata` exemples :

```
{
  "resolutionCategory": "Suivi commande",
  "resolutionNote": "Commande réexpédiée, tracking fourni",
  "feedback": "Service impeccable, merci",
  "transferHistory": [
    {
      "fromAdvisorId": "uuid-xxx",
      "toAdvisorId": "uuid-yyy",
      "reason": "Expertise comptabilité requise",
      "transferredAt": "2024-01-15T14:30:00Z",
      "accepted": true
    }
  ],
  "escalated": false,
  "tags": ["urgent", "commande-perdue"]
}
```

`messages.metadata` exemples :

```
{
  "editedAt": "2024-01-15T10:35:00Z",
  "originalContent": "Texte avant modification",
  "clientContext": {
    "lastOrderId": "uuid-order",
    "lastOrderStatus": "shipped"
  }
}
```

8. API et Interface

Documentation API REST complète :

Authentification :

```
POST /api/auth/login
Body: { email: string, password: string }
Response 200: {
 success: true,
 data: {
   accessToken: string,  // JWT expire 1h
   refreshToken: string, // expire 30j
   user: {
     id: string,
     email: string,
     name: string,
     role: "CLIENT" | "ADVISOR" | "ADMIN",
     companyId: string | null
   }
 }
}
Response 401: { success: false, error: "Invalid credentials" }
Response 429: { success: false, error: "Too many login attempts" }

POST /api/auth/refresh
Headers: { Authorization: "Bearer {refreshToken}" }
Response 200: {
 success: true,
 data: {
   accessToken: string,
   refreshToken: string
 }
}
Response 401: { success: false, error: "Invalid or expired refresh token" }
```

```
POST /api/auth/logout
Headers: { Authorization: "Bearer {accessToken}" }
Response 200: { success: true, message: "Logged out successfully" }

Conversations :
GET /api/conversations
Headers: { Authorization: "Bearer {accessToken}" }
Query params:
 - status: "WAITING" | "ACTIVE" | "RESOLVED" | "CLOSED" (optionnel)
 - page: number (default 1)
 - perPage: number (default 20, max 50)

Response 200: {
 success: true,
 data: [
   {
     id: string,
     clientId: string,
     clientName: string,
     advisorId: string | null,
     advisorName: string | null,
     subject: string | null,
     status: string,
     lastMessage: {
       content: string,
       sentAt: string,
       senderName: string
     },
     unreadCount: number,
     createdAt: string,
     updatedAt: string
   }
```

```
 ],
 meta: {
   pagination: {
     page: 1,
     perPage: 20,
     total: 45,
     totalPages: 3
   }
 }
}
Response 401: { success: false, error: "Unauthorized" }

POST /api/conversations
Headers: { Authorization: "Bearer {accessToken}" }
Body: {
 subject: string (max 200 chars, optionnel),
 initialMessage: string (max 1000 chars, requis)
}

Response 201: {
 success: true,
 data: {
   id: string,
   clientId: string,
   status: "WAITING",
   subject: string,
   createdAt: string,
   messages: [
     {
       id: string,
       content: string,
       senderId: string,
       sentAt: string,
```

```
      type: "TEXT"
    }
  ]
 }
}
Response 400: {
 success: false,
 error: "Validation failed",
 details: [
   { field: "initialMessage", message: "Required field" }
 ]
}
Response 403: {
 success: false,
 error: "Maximum 3 active conversations allowed"
}

GET /api/conversations/:id
Headers: { Authorization: "Bearer {accessToken}" }

Response 200: {
 success: true,
 data: {
   id: string,
   client: { id: string, name: string, email: string, companyName: string },
   advisor: { id: string, name: string } | null,
   subject: string,
   status: string,
   rating: number | null,
   createdAt: string,
   updatedAt: string,
   closedAt: string | null,
   metadata: {
```

```
    resolutionCategory: string,
    resolutionNote: string,
    feedback: string
  }
 }
}
Response 404: { success: false, error: "Conversation not found" }
Response 403: { success: false, error: "Access denied" }

PATCH /api/conversations/:id
Headers: { Authorization: "Bearer {accessToken}" }
Body: {
 status: "RESOLVED" | "CLOSED" (optionnel),
 resolutionCategory: string (requis si status=RESOLVED),
 resolutionNote: string (requis si status=RESOLVED, 50-500 chars),
 rating: number 1-5 (optionnel, client only)
}

Response 200: {
 success: true,
 data: { id: string, status: string, updatedAt: string }
}
Response 400: { success: false, error: "Validation failed" }
Response 403: { success: false, error: "Only assigned advisor can close
conversation" }

Messages :
GET /api/conversations/:conversationId/messages
Headers: { Authorization: "Bearer {accessToken}" }
Query params:
 - page: number (default 1)
 - perPage: number (default 50, max 100)
 - since: string UUID (optionnel, retourne messages > messageId pour sync)
```

```
Response 200: {
 success: true,
 data: [
   {
     id: string,
     conversationId: string,
     senderId: string,
     senderName: string,
     senderRole: "CLIENT" | "ADVISOR",
     content: string,
     type: "TEXT" | "SYSTEM",
     sentAt: string,
     readAt: string | null
   }
 ],
 meta: {
   pagination: { page: 1, perPage: 50, total: 120 },
   hasMore: boolean
 }
}

POST /api/conversations/:conversationId/messages
Headers: { Authorization: "Bearer {accessToken}" }
Body: {
 content: string (max 5000 chars, requis),
 tempId: string (optionnel, pour correlation optimistic update)
}

Response 201: {
 success: true,
 data: {
   id: string,
```

```
   conversationId: string,
   senderId: string,
   content: string,
   sentAt: string,
   tempId: string
 }
}
Response 400: { success: false, error: "Message content required" }
Response 429: { success: false, error: "Rate limit exceeded: 20
messages/minute" }
```

Utilisateurs :

```
GET /api/users/me
Headers: { Authorization: "Bearer {accessToken}" }

Response 200: {
 success: true,
 data: {
   id: string,
   email: string,
   name: string,
   role: string,
   company: {
     id: string,
     name: string,
     siret: string
   } | null,
   createdAt: string
 }
}

GET /api/users/:userId/orders
```

```
Headers: { Authorization: "Bearer {accessToken}" }
Note: Accessible uniquement par conseillers pour contexte client

Response 200: {
 success: true,
 data: [
   {
     id: string,
     orderNumber: string,
     amount: number,
     status: string,
     deliveryDate: string | null,
     createdAt: string
   }
 ]
}
Response 403: { success: false, error: "Advisors only" }
```

Métriques (Conseillers uniquement) :

```
GET /api/metrics
Headers: { Authorization: "Bearer {accessToken}" }
Query params:
 - period: "today" | "week" | "month" (default "today")
 - advisorId: string UUID (optionnel, filtre conseiller spécifique)

Response 200: {
 success: true,
 data: {
   activeConversations: number,
   waitingConversations: number,
   resolvedToday: number,
   averageResponseTime: number, // secondes
```

```
  averageResolutionTime: number, // minutes
  satisfactionRating: number, // moyenne 1-5
  totalRatings: number,
  conversationsByStatus: {
    WAITING: number,
    ACTIVE: number,
    RESOLVED: number,
    CLOSED: number
  },
  advisorStats: [
    {
      advisorId: string,
      advisorName: string,
      activeConversations: number,
      resolvedToday: number,
      averageRating: number
    }
  ]
 }
}
Response 403: { success: false, error: "Advisors only" }
```

Health Check :

```
GET /api/health
Public endpoint (pas d'authentification)

Response 200: {
 success: true,
 data: {
   status: "healthy",
   timestamp: "2024-01-15T10:30:00Z",
   services: {
```

```
    database: "connected",
    redis: "connected",
    socketio: "running"
  },
  version: "1.0.0"
 }
}
Response 503: {
 success: false,
 error: "Service unavailable",
 data: {
  status: "unhealthy",
  services: {
    database: "disconnected",
    redis: "connected",
    socketio: "running"
  }
 }
}
```

Événements Socket.IO détaillés :
Connexion initiale :

```
// Client émet lors connexion
socket.auth = { token: "Bearer eyJxxx..." };
socket.connect();

// Serveur émet après authentification réussie
socket.emit('connection_ack', {
 userId: "uuid-xxx",
 userName: "Jean Dupont",
 role: "ADVISOR",
 activeConversations: ["conv-id-1", "conv-id-2"]
});

// Serveur émet si authentification échouée
socket.emit('error', {
 code: "AUTH_FAILED",
 message: "Invalid or expired token"
});
socket.disconnect();
```

Événements messages :

```
// Client émet envoi message
socket.emit('message:send', {
 conversationId: "uuid-conv",
 content: "Votre commande sera expédiée demain",
 tempId: "temp-uuid-123"
});

// Serveur broadcast aux participants conversation
socket.to(`conversation:${conversationId}`).emit('message:receive', {
 message: {
   id: "uuid-msg",
   conversationId: "uuid-conv",
   senderId: "uuid-user",
   senderName: "Jean Dupont",
   senderRole: "ADVISOR",
   content: "Votre commande sera expédiée demain",
   type: "TEXT",
   sentAt: "2024-01-15T10:30:00Z",
   readAt: null
 },
 tempId: "temp-uuid-123"
});

// Client émet marquage message lu
socket.emit('message:read', {
 conversationId: "uuid-conv",
 messageIds: ["uuid-msg-1", "uuid-msg-2"]
});

// Serveur broadcast confirmation lecture
socket.to(`conversation:${conversationId}`).emit('messages:read', {
```

```
 conversationId: "uuid-conv",
 messageIds: ["uuid-msg-1", "uuid-msg-2"],
 readBy: "uuid-user",
 readAt: "2024-01-15T10:31:00Z"
});
```

Événements typing indicators :

```
// Client émet début frappe
socket.emit('typing:start', {
 conversationId: "uuid-conv"
});

// Serveur broadcast typing indicator (throttle 1s Redis)
socket.to(`conversation:${conversationId}`).emit('typing', {
 conversationId: "uuid-conv",
 userId: "uuid-user",
 userName: "Jean Dupont",
 isTyping: true
});

// Client émet arrêt frappe
socket.emit('typing:stop', {
 conversationId: "uuid-conv"
});

// Serveur broadcast arrêt typing
socket.to(`conversation:${conversationId}`).emit('typing', {
 conversationId: "uuid-conv",
 userId: "uuid-user",
 isTyping: false
});
```

Événements conversations :

```
// Client conseiller émet prise en charge
socket.emit('conversation:claim', {
 conversationId: "uuid-conv"
});

// Serveur broadcast update si succès
io.to(`conversation:${conversationId}`).emit('conversation:update', {
 conversationId: "uuid-conv",
 status: "ACTIVE",
 advisor: {
   id: "uuid-advisor",
   name: "Jean Dupont"
 },
 claimedAt: "2024-01-15T10:30:00Z"
});

// Serveur broadcast aux autres conseillers (retrait liste attente)
io.to('advisors').emit('conversation:claimed', {
 conversationId: "uuid-conv",
 advisorId: "uuid-advisor"
});

// Client conseiller émet transfert
socket.emit('conversation:transfer', {
 conversationId: "uuid-conv",
 toAdvisorId: "uuid-advisor-2",
 reason: "Expertise comptabilité requise"
});

// Serveur émet notification conseiller cible
io.to(`user:${toAdvisorId}`).emit('conversation:transfer_request', {
```

```
 conversationId: "uuid-conv",
 fromAdvisor: {
   id: "uuid-advisor-1",
   name: "Jean Dupont"
 },
 reason: "Expertise comptabilité requise",
 expiresAt: "2024-01-15T10:32:00Z" // 2 min timeout
});

// Conseiller cible accepte transfert
socket.emit('conversation:transfer_accept', {
 conversationId: "uuid-conv"
});

// Serveur broadcast update participants
io.to(`conversation:${conversationId}`).emit('conversation:update', {
 conversationId: "uuid-conv",
 advisor: {
   id: "uuid-advisor-2",
   name: "Marie Martin"
 },
 transferredAt: "2024-01-15T10:31:00Z"
});

// Client conseiller émet clôture
socket.emit('conversation:close', {
 conversationId: "uuid-conv",
 resolutionCategory: "Suivi commande",
 resolutionNote: "Commande réexpédiée avec tracking"
});

// Serveur broadcast clôture
io.to(`conversation:${conversationId}`).emit('conversation:update', {
```

```
 conversationId: "uuid-conv",
 status: "RESOLVED",
 closedAt: "2024-01-15T10:35:00Z"
});
```

Événements notifications :

```
// Serveur émet notification nouvelle conversation en attente
io.to('advisors').emit('notification', {
 type: "NEW_CONVERSATION",
 conversationId: "uuid-conv",
 client: {
   id: "uuid-client",
   name: "Entreprise ABC",
   subject: "Problème livraison"
 },
 createdAt: "2024-01-15T10:30:00Z"
});

// Serveur émet notification conseiller hors ligne
io.to(`user:${clientId}`).emit('notification', {
 type: "ADVISOR_OFFLINE",
 message: "Nos conseillers sont actuellement indisponibles. Nous vous
répondrons dès que possible."
});
```

Gestion erreurs Socket.IO :

```
// Serveur émet erreur validation
socket.emit('error', {
 code: "VALIDATION_ERROR",
 message: "Message content cannot be empty",
 field: "content"
});

// Serveur émet erreur rate limit
socket.emit('error', {
 code: "RATE_LIMIT_EXCEEDED",
 message: "Too many messages sent. Please wait 1 minute.",
 retryAfter: 60 // secondes
});

// Serveur émet erreur métier
socket.emit('error', {
 code: "CONVERSATION_ALREADY_CLAIMED",
 message: "This conversation has been claimed by another advisor",
 conversationId: "uuid-conv"
});
```

Rate limiting Socket.IO :

```
Implémentation middleware rate limit événements :
// backend/src/middleware/socketRateLimit.js
const rateLimitMap = new Map();

const socketRateLimit = (event, maxRequests, windowMs) => {
 return (socket, next) => {
   const key = `${socket.userId}:${event}`;
   const now = Date.now();

   if (!rateLimitMap.has(key)) {
     rateLimitMap.set(key, { count: 1, resetTime: now + windowMs });
     return next();
   }

   const limiter = rateLimitMap.get(key);

   if (now > limiter.resetTime) {
     limiter.count = 1;
     limiter.resetTime = now + windowMs;
     return next();
   }

   if (limiter.count >= maxRequests) {
     socket.emit('error', {
       code: "RATE_LIMIT_EXCEEDED",
       message: `Maximum ${maxRequests} ${event} per ${windowMs/1000}s`,
       retryAfter: Math.ceil((limiter.resetTime - now) / 1000)
     });
     return;
   }
```

```
   limiter.count++;
   next();
 };
};

// Utilisation
io.on('connection', (socket) => {
 socket.on('message:send',
   socketRateLimit('message:send', 20, 60000), // 20 msg/min
   handleMessageSend
 );
  socket.on('typing:start',
   socketRateLimit('typing:start', 30, 60000), // 30/min
   handleTypingStart
 );
});
```

9. Règles Métier

Règles conversations

1. **Création conversation :**

    - Client authentifié peut créer nouvelle conversation si < 3 conversations actives simultanées
    - Message initial obligatoire minimum 10 caractères maximum 1000 caractères
    - Sujet optionnel maximum 200 caractères
    - Conversation créée avec statut WAITING automatiquement
    - Notification broadcast immédiate tous conseillers disponibles
    - Si aucun conseiller connecté : message automatique "Nos conseillers reviendront sous peu"
2. **Prise en charge conversation :**

    - Seuls conseillers rôle ADVISOR peuvent prendre conversations en charge
    - Conseiller peut gérer maximum 4 conversations actives simultanément
    - Conseiller doit avoir statut "disponible" (pas en pause)
    - Prise en charge atomic via transaction base données évitant double assignation
    - Si race condition détectée : erreur retournée premier conseiller informé
    - Message système automatique inséré timeline : "[Conseiller X] vous répond"
    - Conversation passe statut ACTIVE avec timestamp claimedAt
3. **Transfert conversation :**

    - Seul conseiller actuellement assigné peut transférer conversation
    - Conseiller cible doit avoir < 4 conversations actives et être connecté
    - Raison transfert obligatoire minimum 20 caractères pour traçabilité
    - Conseiller cible a 2 minutes pour accepter ou refuser transfert
    - Si accepté : conversation réassignée avec insertion message système
    - Si refusé : conversation retourne conseiller initial avec notification
    - Si timeout 2 minutes : conversation retourne pool WAITING avec broadcast conseillers
    - Maximum 2 transferts par conversation évitant ping-pong
    - Historique transferts stocké metadata JSON pour audit qualité
4. **Clôture conversation :**

    - Seul conseiller assigné peut marquer conversation résolue
    - Note résolution obligatoire 50-500 caractères pour traçabilité qualité
    - Catégorie résolution obligatoire parmi liste prédéfinie
    - Conversation passe statut RESOLVED avec timestamp closedAt
    - Client invité immédiatement noter satisfaction via modal 1-5 étoiles
    - Si client note : conversation passe statut CLOSED automatiquement

    - Si client ignore 48h : cron job transition automatique RESOLVED vers CLOSED
    - Conversations CLOSED non modifiables sauf réouverture < 7 jours
5. **Réouverture conversation :**

    - Client peut rouvrir conversation CLOSED si < 7 jours depuis clôture
    - Réouverture crée nouvelle conversation liée ancienne via metadata.parentConversationId
    - Ancien historique accessible en lecture seule via lien "Voir conversation précédente"
    - Si > 7 jours : bouton réouverture disabled avec tooltip "Créez une nouvelle conversation"
    - Maximum 1 réouverture par conversation originale

Règles messages

1. **Envoi messages :**

    - Utilisateur authentifié peut envoyer messages uniquement conversations où il est participant
    - Client participant si conversation.clientId = userId
    - Conseiller participant si conversation.advisorId = userId
    - Contenu message obligatoire minimum 1 caractère maximum 5000 caractères
    - Sanitization HTML automatique via DOMPurify évitant injection XSS
    - Rate limit 20 messages par minute par utilisateur
    - Messages type SYSTEM réservés serveur backend uniquement
    - Timestamp sentAt généré serveur évitant falsification client
2. **Lecture messages :**

    - Marquage automatique messages lus si conversation ouverte et tab navigateur focus
    - Event message:read envoyé batch toutes les 3 secondes pour optimisation
    - Champ readAt nullable initialement, peuplé lors première lecture
    - Compteur unread mis à jour temps réel via Socket.IO
    - Messages marqués lus si destinataire voit message dans viewport scroll
3. **Édition/suppression messages :**

    - Phase MVP : pas d'édition ni suppression messages autorisée
    - Messages persistés immuables pour traçabilité légale B2B
    - Phase 2 possible : édition < 5 minutes avec historique metadata.editedAt

Règles utilisateurs

1. **Authentification :**
    - Access token JWT expire 1 heure après émission
    - Refresh token expire 30 jours après émission
    - Refresh automatique silent access token 5 minutes avant expiration
    - Si refresh échoue : déconnexion forcée avec redirect login
    - Maximum 5 tentatives login échouées par IP toutes les 15 minutes
2. **Rôles et permissions :**
    - CLIENT : peut créer conversations, envoyer messages, noter satisfaction
    - ADVISOR : peut prendre en charge conversations, envoyer messages, transférer, clôturer, accéder historique commandes clients
    - ADMIN : tous droits ADVISOR + accès métriques globales + gestion utilisateurs
    - Vérification rôle middleware backend chaque requête sensible
    - Row Level Security PostgreSQL filtre automatiquement données accessibles
3. **Statut disponibilité conseillers :**
    - Conseiller connecté avec socket actif = disponible automatiquement
    - Conseiller peut passer statut "en pause" manuellement bloquant nouvelles assignations
    - Conversations actives conservées pendant pause
    - Déconnexion socket automatiquement passe statut indisponible
    - Heartbeat Socket.IO toutes les 25 secondes détecte déconnexions

Règles métriques et qualité

1. **Calcul temps réponse :**
    - Temps réponse = timestamp premier message conseiller - timestamp création conversation
    - Exclu conversations créées hors horaires ouvrés (calcul démarre 8h lendemain)
    - Objectif < 3 minutes pendant horaires ouvrés
    - Alerte dashboard si moyenne journalière > 5 minutes
2. **Calcul temps résolution :**
    - Temps résolution = timestamp clôture - timestamp création conversation
    - Moyenne cible 15 minutes conversations simples
    - Exclut temps conversations statut WAITING avant prise en charge
3. **Satisfaction client :**
    - Moyenne rating 1-5 étoiles toutes conversations notées
    - Objectif > 4.0/5 global
    - Alerte si conseiller individuel < 3.5/5 sur 20 conversations minimum

    - Rating optionnel mais taux réponse cible 60%
4. **Audit trail :**

    - Tous events majeurs loggés Winston JSON structuré :
        - Création/prise en charge/transfert/clôture conversations
        - Tentatives authentification échouées
        - Erreurs serveur 500
    - Logs persistés 90 jours minimum
    - Extraction possible pour analyse RGPD

10. Gestion des Performances

Stratégies optimisation frontend :

1. **Lazy loading composants :**

```
 // React.lazy() pour routes non critiques
const ConversationHistory = React.lazy(() =>
  import('./components/ConversationHistory')
);

// Suspense fallback spinner
<Suspense fallback={<LoadingSpinner />}>
  <ConversationHistory />
</Suspense>
```

2. **Virtualisation listes messages :**
   - Implémentation react-window pour conversations > 100 messages
   - Rendu uniquement messages visibles viewport + buffer 10 messages
   - Réduction consommation mémoire 80% pour longues conversations
   - Scroll performance maintenue 60fps
3. **Optimistic updates :**
   - Messages affichés immédiatement interface avec tempId avant confirmation serveur
   - Réduction latence perçue utilisateur
   - Rollback automatique si erreur serveur retournée
   - État synchronisé via React Query mutations

Debouncing typing indicators :

```
 const handleTyping = useMemo(
  () => debounce(() => {
    socket.emit('typing:start', { conversationId });
  }, 300),
  [conversationId]
);
```

4.
   - Évite flood événements Socket.IO chaque frappe clavier
   - Throttle 300ms optimal balance réactivité/performance
5. **Pagination messages :**
   - Chargement initial 50 derniers messages conversation
   - Infinite scroll vers ancien historique charge 50 messages supplémentaires
   - Prefetching next page démarré lorsque utilisateur atteint 80% scroll
   - Cache React Query évite rechargements inutiles

6. **Code splitting :**

    - Bundle principal < 200KB gzipped
    - Chunks séparés par route et composants lourds
    - Preload chunks critiques via `<link rel="preload">`

Stratégies optimisation backend :
Pooling connexions PostgreSQL :

```
 // Prisma configuration
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Connection pool params
DATABASE_URL="postgresql://user:pass@host:5432/db?
  connection_limit=10&
  pool_timeout=20&
  connect_timeout=10"
```

1.
    - Pool 10 connexions simultanées suffisant volume B2B
    - Timeout 20 secondes évite blocage requêtes lentes

Index base données critiques :

```
 -- Index composite conversations statut + date
CREATE INDEX idx_conv_status_created
  ON conversations(status, created_at DESC);

-- Index messages pagination
CREATE INDEX idx_messages_conv_sent
  ON messages(conversation_id, sent_at DESC);

-- Index user lookups
CREATE INDEX idx_users_email ON users(email);
```

2.
    - Réduction temps requête 90% requêtes fréquentes
    - EXPLAIN ANALYZE validation plans exécution

3. **Caching Redis stratégique :**

    - Typing indicators TTL 5 secondes évitant surcharge PostgreSQL
    - Présence utilisateurs connectés TTL 30 secondes
    - Sessions actives Socket.IO si mode multi-serveurs
    - Pas de cache conversations/messages conservant PostgreSQL source vérité

Requêtes optimisées Prisma :

```
// Include relations sélectif évitant over-fetching
const conversation = await prisma.conversation.findUnique({
  where: { id },
  include: {
    client: {
      select: { id: true, name: true, email: true, company: { select: { name: true }}}
    },
    advisor: {
      select: { id: true, name: true }
    },
    messages: {
      take: 50,
      orderBy: { sentAt: 'desc' },
      include: {
        sender: {
          select: { id: true, name: true, role: true }
        }
      }
    }
  }
});
```

4.
    - Select explicite colonnes nécessaires évite SELECT *
    - Pagination intégrée Prisma via take/skip

Compression réponses HTTP :

```
const compression = require('compression');
app.use(compression({
  level: 6, // Balance compression ratio / CPU
  threshold: 1024 // Compresse seulement > 1KB
}));
```

5.
    - Réduction taille payloads JSON 60-70%
    - Latence réseau divisée par 2-3

Rate limiting intelligent :

```
const rateLimit = require('express-rate-limit');

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 min
  max: 100, // Max 100 requêtes
  standardHeaders: true,
```

```
  legacyHeaders: false,
  skip: (req) => req.user?.role === 'ADMIN' // Exempte admins
});

app.use('/api/', apiLimiter);
```

6.

Métriques performance cibles :

Copier le tableau

Métrique	Objectif	Méthode mesure
Time to First Byte (TTFB)	< 200ms	Lighthouse/WebPageTest
First Contentful Paint	< 1.5s	Lighthouse
Largest Contentful Paint	< 2.5s	Lighthouse
Time to Interactive	< 3s	Lighthouse
Latence WebSocket ping	< 100ms	Socket.IO diagnostics
Temps réponse API GET	< 300ms P95	APM monitoring
Temps réponse API POST	< 500ms P95	APM monitoring
Messages/seconde supportés	50 msg/s	Load testing k6
Connexions WebSocket simultanées	200	Load testing Artillery
Utilisation CPU serveur	< 60% moyenne	Railway metrics
Utilisation RAM serveur	< 70% max	Railway metrics
Taille bundle JS principal	< 200KB gzip	Webpack analyzer

Load testing scénarios :

Script k6 test charge :

```
import http from 'k6/http';
import ws from 'k6/ws';
import { check, sleep } from 'k6';

export const options = {
 stages: [
```

```
    { duration: '2m', target: 50 },  // Ramp-up 50 utilisateurs
    { duration: '5m', target: 50 },  // Plateau 50 users
    { duration: '2m', target: 100 }, // Spike 100 users
    { duration: '3m', target: 100 }, // Plateau spike
    { duration: '2m', target: 0 },   // Ramp-down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% requests < 500ms
    ws_connecting: ['p(95)<1000'],    // WebSocket connect < 1s
  },
};

export default function () {
  // Login API
  const loginRes = http.post('https://api.example.com/api/auth/login', {
    email: `user-${__VU}@example.com`,
    password: 'test123'
  });
   check(loginRes, { 'login successful': (r) => r.status === 200 });
   const token = JSON.parse(loginRes.body).data.accessToken;
   // WebSocket connection
  const url = `wss://api.example.com?token=${token}`;
  const res = ws.connect(url, {}, function (socket) {
    socket.on('open', () => {
      // Envoi 5 messages
      for (let i = 0; i < 5; i++) {
        socket.send(JSON.stringify({
          event: 'message:send',
          data: {
            conversationId: 'test-conv-id',
            content: `Test message ${i}`
          }
        }));
```

```
      sleep(2);
    }
  });

  socket.setTimeout(() => {
    socket.close();
  }, 30000);
});
 check(res, { 'WebSocket connected': (r) => r && r.status === 101 });
 sleep(1);
}
```

11. Sécurité

Authentification et autorisation :

1. **JWT tokens Néon :**

Access token signé algorithmeavec payload minimal :

```
{
  "sub": "uuid-user",
  "email": "user@example.com",
  "role": "CLIENT",
  "iat": 1705320000,
  "exp": 1705323600
}
```

-
- Refresh token opaque stocké Néon avec rotation automatique
- Signature vérifiée backend via clé publique Néon (RS256)
- Tokens stockés HttpOnly cookies si possible sinon LocalStorage avec fallback

Row Level Security PostgreSQL :

```sql
-- Politique conversations : utilisateur voit uniquement ses conversations
CREATE POLICY conversation_access ON conversations
 FOR SELECT
 USING (
   auth.uid() = client_id
   OR auth.uid() = advisor_id
   OR (SELECT role FROM users WHERE id = auth.uid()) = 'ADMIN'
 );

-- Politique messages : utilisateur voit messages conversations accessibles
CREATE POLICY message_access ON messages
 FOR SELECT
 USING (
   conversation_id IN (
     SELECT id FROM conversations
     WHERE client_id = auth.uid() OR advisor_id = auth.uid()
   )
 );
```

```sql
-- Politique insert messages : utilisateur envoie messages ses conversations
CREATE POLICY message_insert ON messages
 FOR INSERT
 WITH CHECK (
   sender_id = auth.uid()
   AND conversation_id IN (
     SELECT id FROM conversations
     WHERE client_id = auth.uid() OR advisor_id = auth.uid()
   )
 );
```

Middleware autorisation Express :

```javascript
const authenticateJWT = async (req, res, next) => {
 const token = req.headers.authorization?.replace('Bearer ', '');
  if (!token) {
   return res.status(401).json({
     success: false,
     error: 'Authentication required'
   });
 }
  try {
   const { data: { user }, error } = await Néon.auth.getUser(token);

   if (error || !user) {
     return res.status(401).json({
       success: false,
       error: 'Invalid token'
     });
   }

   req.user = user;
```

```
    next();
  } catch (err) {
    return res.status(401).json({
      success: false,
      error: 'Token verification failed'
    });
  }
};

const authorize = (roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        error: 'Insufficient permissions'
      });
    }
    next();
  };
};
```

// Utilisation
app.get('/api/metrics',
  authenticateJWT,
  authorize(['ADVISOR', 'ADMIN']),
  getMetrics
);

2. **Protection contre attaques courantes :**
   1. **XSS (Cross-Site Scripting) :**
      - Sanitization automatique contenu messages via DOMPurify backend avant stockage

CSP headers strictes :
app.use(helmet.contentSecurityPolicy({
  directives: {

```
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'", "'unsafe-inline'"], // Vite dev uniquement
    styleSrc: ["'self'", "'unsafe-inline'"],
    connectSrc: ["'self'", "wss://api.example.com", "https://api.example.com"],
    imgSrc: ["'self'", "data:", "https:"],
    fontSrc: ["'self'"],
    objectSrc: ["'none'"],
    upgradeInsecureRequests: []
  }
}));
```

-
- Échappement HTML automatique React (dangerouslySetInnerHTML interdit)

2. **CSRF (Cross-Site Request Forgery) :**

   - Tokens CSRF pas nécessaires avec JWT stateless dans Authorization header

SameSite cookies si stockage cookie utilisé :

```
res.cookie('refreshToken', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',
  maxAge: 30 * 24 * 60 * 60 * 1000 // 30 jours
});
```

-

3. **SQL Injection :**

   - Prisma ORM parameterized queries automatiques
   - Aucune concaténation SQL brute dans codebase
   - Validation Zod inputs avant passage Prisma
4. **NoSQL Injection (Redis) :**

   - Redis uniquement cache éphémère, pas de requêtes utilisateur direct

Clés Redis préfixées et sanitizées :

```
const sanitizeKey = (key) => key.replace(/[^a-zA-Z0-9:-]/g, '');
const redisKey = `typing:${sanitizeKey(conversationId)}:${sanitizeKey(userId)}`;
```

-

5. **DDoS et rate limiting :**

   - Rate limiting global : 100 requêtes API / 15 min / IP
   - Rate limiting login : 5 tentatives / 15 min / IP
   - Rate limiting Socket.IO : 20 messages / min / utilisateur
   - Cloudflare optionnel devant infrastructure si attaques persistantes

6. **Mass Assignment :**

Whitelisting explicite champs autorisés :

```
const updateConversation = async (req, res) => {
  const allowedFields = ['status', 'resolutionCategory', 'resolutionNote'];
  const data = pick(req.body, allowedFields);

  // Prisma update avec data sanitizé
  await prisma.conversation.update({
    where: { id: req.params.id },
    data
  });
};
```

-

Chiffrement données :

1. **Données en transit :**
   - HTTPS/TLS 1.3 obligatoire toutes communications
   - WSS (WebSocket Secure) pour Socket.IO
   - Certificates Let's Encrypt automatiques via PaaS ou Certbot

HSTS header force HTTPS :

```
app.use(helmet.hsts({
  maxAge: 31536000, // 1 an
  includeSubDomains: true,
  preload: true
}));
```

-

2. **Données au repos :**
   - PostgreSQL Néon chiffrement AES-256 au repos natif
   - Pas de chiffrement applicatif supplémentaire (overhead performance)
   - Backups automatiques chiffrés Néon
3. **Secrets management :**
   - Variables environnement jamais committées Git
   - `.env` fichier dans `.gitignore`
   - Secrets managés plateforme déploiement (Railway secrets, Vercel env vars)
   - Rotation secrets sensibles (JWT secret, DB password) tous les 90 jours

Conformité RGPD :

1. **Minimisation données :**
   - Collecte uniquement données nécessaires fonctionnement service
   - Pas de tracking analytics tiers MVP (Google Analytics exclu)

    - Cookies strictement fonctionnels (authentification uniquement)
2. **Consentement :**

    - Banner cookies affichant politique utilisation
    - Pas de cookies marketing/tracking donc pas de wall consent complexe
    - Checkbox acceptation CGU lors création compte
3. **Droit accès et portabilité :**

Endpoint API exportation données utilisateur :
GET /api/users/me/export
Response: ZIP contenant JSON conversations + messages
    -
4. **Droit à l'oubli :**

    - Procédure suppression compte via interface ou demande écrite
    - Soft delete initial 30 jours rétention puis hard delete définitif
    - Anonymisation messages utilisateur supprimé (remplace nom par "Utilisateur supprimé")
5. **Conservation données :**

    - Conversations actives/résolues : conservation 3 ans obligation légale B2B
    - Logs serveur : 90 jours
    - Données utilisateurs inactifs > 2 ans : notification puis suppression automatique

Audit sécurité :
Checklist pré-déploiement production :

- Scan vulnérabilités npm audit et Snyk
- Revue headers sécurité via securityheaders.com
- Test pénétration manuel injection basique (OWASP Top 10)
- Vérification exposition endpoints sensibles non authentifiés
- Test rate limiting efficace via script automatisé
- Validation certificates SSL/TLS via ssllabs.com
- Revue permissions base données (pas de user root exposé)
- Test Row Level Security PostgreSQL avec différents rôles
- Vérification secrets environnement non exposés logs
- Documentation procédure incident sécurité (contact, escalation)

12. Observabilité

Logging structuré :
Configuration Winston :

```
const winston = require('winston');


const logger = winston.createLogger({
 level: process.env.LOG_LEVEL || 'info',
```

```
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: {
    service: 'chat-api',
    environment: process.env.NODE_ENV
  },
  transports: [
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.simple()
      )
    }),
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error'
    }),
    new winston.transports.File({
      filename: 'logs/combined.log'
    })
  ]
});

// Exemple log structuré
logger.info('Conversation claimed', {
  conversationId: 'uuid-xxx',
  advisorId: 'uuid-yyy',
  clientId: 'uuid-zzz',
  waitingTime: 120, // secondes
  timestamp: new Date().toISOString()
```

```
});
```

Événements loggés prioritaires :

- Authentification success/failure avec IP source
- Création/prise en charge/transfert/clôture conversations
- Erreurs serveur 500 avec stack trace complète
- Rate limit dépassé avec utilisateur concerné
- Déconnexions WebSocket anormales
- Requêtes API lentes > 1 seconde
- Tentatives accès non autorisé (403)

Monitoring erreurs Sentry :

Configuration Sentry Node.js :

```
const Sentry = require('@sentry/node');
const Tracing = require('@sentry/tracing');

Sentry.init({
 dsn: process.env.SENTRY_DSN,
 environment: process.env.NODE_ENV,
 tracesSampleRate: 0.1, // 10% transactions tracées
 integrations: [
   new Sentry.Integrations.Http({ tracing: true }),
   new Tracing.Integrations.Express({ app }),
   new Tracing.Integrations.Prisma({ client: prisma })
 ],
 beforeSend(event, hint) {
   // Filtre erreurs bruit (404, timeouts clients)
   if (event.exception) {
     const error = hint.originalException;
     if (error.status === 404 || error.code === 'ECONNRESET') {
       return null;
     }
   }
   return event;
 }
});
```

```
// Middleware Express
app.use(Sentry.Handlers.requestHandler());
app.use(Sentry.Handlers.tracingHandler());

// Routes...

app.use(Sentry.Handlers.errorHandler());

Configuration Sentry React :
import * as Sentry from '@sentry/react';
import { BrowserTracing } from '@sentry/tracing';

Sentry.init({
 dsn: import.meta.env.VITE_SENTRY_DSN,
 environment: import.meta.env.MODE,
 integrations: [
   new BrowserTracing(),
   new Sentry.Replay({
     maskAllText: true,
     blockAllMedia: true
   })
 ],
 tracesSampleRate: 0.1,
 replaysSessionSampleRate: 0.1,
 replaysOnErrorSampleRate: 1.0 // Replay complet si erreur
});
```

Métriques application :
Métriques exposées endpoint `/api/metrics` (Prometheus compatible) :

- `chat_active_connections_total` : connexions WebSocket actives
- `chat_messages_sent_total` : compteur messages envoyés (labels: user_role)
- `chat_conversations_created_total` : compteur conversations créées

- `chat_conversations_active_gauge` : gauge conversations statut ACTIVE
- `chat_response_time_seconds` : histogram temps réponse conseiller
- `chat_resolution_time_seconds` : histogram temps résolution total
- `http_request_duration_seconds` : histogram latence endpoints API
- `websocket_events_total` : compteur events Socket.IO (labels: event_type)

Implémentation simple sans bibliothèque :

```
const metrics = {
  activeConnections: 0,
  messagesSent: { CLIENT: 0, ADVISOR: 0 },
  conversationsCreated: 0,
  activeConversations: 0
};
```

```
// Incrémentation events
io.on('connection', (socket) => {
 metrics.activeConnections++;
  socket.on('disconnect', () => {
   metrics.activeConnections--;
 });
  socket.on('message:send', (data) => {
   metrics.messagesSent[socket.user.role]++;
 });
});

// Endpoint exposition
app.get('/api/metrics', authenticateJWT, authorize(['ADMIN']), (req, res) => {
 res.json({
   timestamp: new Date().toISOString(),
   metrics
 });
});
```

Monitoring infrastructure :

Services externes monitoring :

- **Uptime Robot** : ping HTTP endpoint `/api/health` toutes les 5 minutes
  - Alerte email/SMS si down > 2 checks consécutifs

        - Page status publique optionnelle
- **Railway/Render Dashboard** : métriques natives CPU/RAM/réseau
- **Néon Dashboard** : métriques base données (connexions actives, queries lentes)
- **Vercel Analytics** : Core Web Vitals frontend

Alertes configurées :

- CPU serveur > 80% pendant 5 minutes
- RAM serveur > 85% pendant 5 minutes
- Endpoint `/api/health` status 503
- Erreurs Sentry rate > 10/minute
- Latence P95 API > 1 seconde
- Connexions PostgreSQL pool saturé

Dashboard temps réel conseilleurs :

Métriques affichées interface conseiller :

- Conversations actives/en attente count
- Temps réponse moyen dernière heure
- Satisfaction moyenne jour
- Graphique conversations créées par heure (dernières 24h)
- Liste conseillers connectés avec charge actuelle

Rafraîchissement :

- Counts mis à jour temps réel via Socket.IO events
- Graphiques rechargés toutes les 5 minutes via polling API
- Cache React Query 5 minutes évite requêtes inutiles

13. Déploiement

Stratégie déploiement recommandée :

Déploiement continu automatisé via GitHub Actions :

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

```yaml
  deploy-backend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Railway
        env:
          RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}
        run: |
          npm install -g @railway/cli
          railway up --service backend

  deploy-frontend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Vercel
        env:
          VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
        run: |
          npm install -g vercel
          vercel --prod --token=$VERCEL_TOKEN

  migrate-database:
    needs: [deploy-backend]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Prisma migrations
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
        run: |
          npm install
          npx prisma migrate deploy
```

Procédure déploiement manuel (fallback) :

1. **Préparation :**
    - Tag release Git : `git tag v1.0.0 && git push --tags`
    - Revue changelog modifications depuis dernière release
    - Vérification tests passent localement : `npm test`
    - Backup base données : `pg_dump via Néon Dashboard`

Déploiement backend :

```
# Build TypeScript
npm run build

# Railway CLI deploy
railway login
railway link [project-id]
railway up

# Vérification santé
curl https://api.example.com/api/health
```

2. **Migrations base données :**

```
# Génération migration si schema modifié
npx prisma migrate dev --name add_metadata_field

# Application production
npx prisma migrate deploy

# Vérification schéma
npx prisma db pull
```

3. **Déploiement frontend :**

```
# Build production
npm run build

# Vercel deploy
vercel --prod

# Vérification Core Web Vitals
lighthouse https://app.example.com --view
```

4.
5. **Tests post-déploiement :**
    - Smoke tests endpoints critiques :
        - `POST /api/auth/login` : authentification fonctionne
        - `GET /api/conversations` : liste conversations accessible
        - Connexion WebSocket établie sans erreur
    - Test conversation end-to-end production :
        - Compte test crée conversation
        - Conseiller test prend en charge

        - Échange 3 messages
        - Clôture conversation
    - Vérification Sentry aucune erreur nouvelle remontée
    - Check métriques CPU/RAM stables après 10 minutes

Rollback procédure :
Si déploiement introduit bug critique :
Rollback backend :

```
# Railway rollback dernière version stable
railway rollback [deployment-id-previous]

# Ou redéploy tag Git précédent
git checkout v0.9.0
railway up
```

1.

Rollback frontend :

```
# Vercel rollback via dashboard ou CLI
vercel rollback [deployment-url-previous]
```

2.
3. **Rollback migrations base données :**

    - Prisma ne supporte pas rollback automatique

Procédure manuelle requise :

```
# Restaurer backup pré-migration
pg_restore --clean --if-exists -d [DATABASE_URL] backup.sql

# Ou migration down custom
psql [DATABASE_URL] < migrations/down/xxx_rollback.sql
```

    -
    - **Prévention** : tester migrations staging avant production

Communication équipes :
Notification déploiement production :
Canal Slack `#deployments` :

```
🚀 Déploiement Production v1.2.0
- Backend : https://api.example.com (Railway)
- Frontend : https://app.example.com (Vercel)
- Changements :
  • Ajout transfert conversations entre conseillers
  • Fix latence chargement historique messages
  • Migration : ajout colonne conversations.metadata
- Tests post-deploy : ✅ Passed
- Rollback si nécessaire : git tag v1.1.0
```

Email équipe support :
Objet : Déploiement nouvelles fonctionnalités chat - 15/01/2024

Bonjour,

Une nouvelle version du chat support a été déployée ce jour à 14h00.

Nouvelles fonctionnalités :
- Vous pouvez maintenant transférer conversations vers collègues via bouton "Transférer"
- Historique messages charge plus rapidement (optimisation backend)

Rien à faire de votre côté, rechargez simplement page si ouverte.

En cas de problème : contacter tech@example.com

Merci,
Équipe Tech

Fenêtre maintenance planifiée :
Si migration base données longue (> 1 minute) :

1. **Planification :**
    - Fenêtre hors pic usage : mardi 6h-7h du matin
    - Annonce 48h avance clients B2B via email + banner interface

Procédure maintenance :

```
# Activer mode maintenance (page statique)
vercel env add MAINTENANCE_MODE true

# Backend continue servir page maintenance
# Pendant ce temps : migration base données
npx prisma migrate deploy

# Tests post-migration
npm run test:integration

# Désactiver mode maintenance
vercel env rm MAINTENANCE_MODE
```

2.

Page maintenance HTML :

```
<!DOCTYPE html>
<html>
<head>
```

```
  <title>Maintenance en cours</title>
  <meta http-equiv="refresh" content="60">
</head>
<body>
  <h1>Maintenance planifiée</h1>
  <p>Le chat support sera de retour sous peu (durée estimée : 30 min)</p>
  <p>Pour urgences : contact@example.com</p>
  <p>Page rechargée automatiquement...</p>
</body>
</html>
```

3.

Impact déploiement :

Déploiement standard (sans migration DB) :

- **Zero-downtime** : Railway/Vercel blue-green deployment automatique
- **Connexions WebSocket actives** : maintenues pendant déploiement grâce health checks
- **Clients connectés** : peuvent continuer conversations sans interruption
- **Nouveaux clients** : routés automatiquement nouvelle version

Déploiement avec migration DB :

- **Downtime potentiel** : 30 secondes - 2 minutes selon complexité migration
- **Connexions WebSocket** : déconnectées puis reconnexion automatique après migration
- **Messages en cours envoi** : bufferisés client puis renvoyés reconnexion
- **Risque perte données** : nul si migration testée staging

Monitoring post-déploiement :

Dashboard surveillance 30 minutes post-deploy :

- Taux erreurs Sentry (doit rester < 0.1%)
- Latence API P95 (doit rester < 500ms)
- Connexions WebSocket actives (retour niveau pré-deploy sous 5 min)
- Logs serveur erreurs 500 (aucune attendue)
- Feedback utilisateurs Slack channel support (écoute proactive)

Checklist validation déploiement réussi :

- Health check endpoint retourne 200 OK
- Sentry aucune erreur nouvelle 15 min post-deploy
- Tests automatisés Playwright passent
- Métriques CPU/RAM normales
- Aucune régression vitesse chargement frontend (Lighthouse score maintenu)

14. Risques

Risques techniques :

Copier le tableau

Risque	Probabilité	Impact	Mitigation	Plan contingence

Panne Néon prolongée	Faible (SLA 99.9%)	Critique (service indisponible)	- Monitoring externe Uptime Robot - Backup quotidien automatique - Communication clients proactive	- Page statique maintenance - Bascule email formulaire temporaire - Restauration backup sur PostgreSQL alternatif si > 2h
Dépassement quota gratuit Néon	Moyenne (si croissance rapide)	Moyen (facturation surprise)	- Alertes 80% quota utilisé - Dashboard monitoring quotidien - Budget max configuré	- Migration plan payant Néon - Ou migration PostgreSQL self-hosted Railway
Faille sécurité XSS/SQL injection	Faible	Critique (fuite données)	- Audit code pré-production - npm audit régulier - Sanitization systématique inputs	- Patch correctif immédiat - Rotation secrets API - Communication clients transparente RGPD
Saturation connexions WebSocket	Moyenne (si pic usage inattendu)	Moyen (nouveaux clients bloqués)	- Load testing k6 avant lancement - Auto-scaling Railway activé - Limite 200 connexions monitoring	- Scaling horizontal backend - Queue conversations en attente - Message "Forte affluence, patientez..."
Bug critique post-déploiement	Moyenne	Élevé (régression fonctionnelle)	- Tests automatisés CI/CD - Staging environnement réplique prod - Feature flags pour rollback partiel	- Rollback version précédente < 15 min - Hotfix prioritaire - Post-mortem analyse cause

Risques métier :
Copier le tableau

Risque	Probabilité	Impact	Mitigation	Plan contingence

Adoption faible clients B2B	Moyenne	Élevé (ROI négatif)	- Pilote 50 clients early adopters sélectionnés - Communication avantages (réponse 10x plus rapide) - Formation clients onboarding guidé - Coexistence email/chat 6 mois	- Interviews qualitatives obstacles adoption - Ajustements UX selon feedbacks - Incentives utilisation (réponse prioritaire chat)
Résistance conseillers changement	Faible	Moyen (sous-utilisation outil)	- Formation 2h conseillers pré-lancement - Implication tests UAT phase développement - Dashboard métriques valorisant travail	- Sessions feedback régulières - Ajustements interface selon retours - Communication management soutien
Augmentation charge travail conseillers	Moyenne	Moyen (burnout équipe)	- Limite 4 conversations simultanées par conseiller - Métriques temps réponse monitoring surchage - Recrutement 2 conseillers additionnels prévu T2	- Réduction temporaire heures ouverture chat - Priorisation conversations urgentes - Passage mode asynchrone si débordement
Attentes clients insatisfaites (réponse instantanée 24/7)	Moyenne	Moyen (déception)	- Communication claire horaires disponibilité - Message automatique hors horaires avec délai réponse - SLA 3 minutes affiché interface	- Ajustement communication attentes réalistes - Extension horaires progressivement si demande - Email fallback toujours disponible

Risques légaux et conformité :
Copier le tableau

Risque	Probabilité	Impact	Mitigation

Non-conformité RGPD	Faible	Critique (amende jusqu'à 4% CA)	- Audit RGPD pré-lancement - DPO consulté architecture - Privacy by design intégré
Perte données conversations (obligation légale 3 ans)	Faible	Élevé (sanction légale)	- Backups automatiques quotidiens Néon - Réplication géographique données - Tests restauration trimestriels

Risques opérationnels :
Copier le tableau

Risque	Probabilité	Impact	Mitigation
Dépendance développeur unique (bus factor)	Moyenne	Élevé (blocage projet si départ)	- Documentation technique exhaustive - Code review systématique - Pair programming sessions clés
Dérive planning MVP (6 semaines)	Élevée	Moyen (retard lancement)	- Priorisation stricte fonctionnalités P0 - Sprints 1 semaine avec démos - Buffer 1 semaine intégré timeline

Matrice probabilité/impact globale :
Impact

```
↑
│ Critique │ Panne Néon      │                  │
│          │ Faille sécurité │                  │
├──────────┼─────────────────┼──────────────────┤
│ Élevé    │ Adoption faible │ Bug post-deploy  │
│          │ Perte données   │                  │
├──────────┼─────────────────┼──────────────────┤
│ Moyen    │ Résistance conseillers│ Saturation WebSocket│
│          │ Attentes clients │ Surcharge travail │
├──────────┼─────────────────┼──────────────────┤
│ Faible   │                 │ Dérive planning  │
│          │                 │                  │
└──────────┴─────────────────┴──────────────────→
     Faible      Moyenne        Élevée
                Probabilité
```

Indicateurs alerte précoce :
Métriques surveillance risques :

- **Adoption** : taux utilisation chat vs email suivi hebdomadaire (objectif > 60% sous 2 mois)
- **Satisfaction** : rating moyen < 3.5/5 déclenche alerte escalade management
- **Performance** : latence P95 > 1s ou taux erreur > 1% alerte technique
- **Charge conseillers** : temps résolution moyen > 20 min alerte surcharge
- **Sécurité** : scan vulnérabilités npm hebdomadaire automatisé

15. Questions Ouvertes

Questions architecture :

1. **Redis nécessaire MVP ou différable ?**
    - **Contexte** : Redis prévu caching typing indicators et présence utilisateurs
    - **Trade-off** :
        - Avec Redis : latence typing indicators < 100ms, scaling multi-serveurs facilité
        - Sans Redis : stockage éphémère mémoire Node.js suffisant volume B2B, simplicité architecture
    - **Décision recommandée** : **Différer phase 2**. Volume B2B permet stockage mémoire serveur unique. Ajout Redis si migration multi-serveurs ou latence problématique pilote.
2. **Architecture monolithe vs microservices ?**
    - **Contexte** : actuellement monolithe Node.js unique
    - **Trade-off** :
        - Monolithe : simplicité développement/déploiement, latence minimale, coût infrastructure réduit
        - Microservices : scalabilité indépendante, isolation pannes, complexité accrue
    - **Décision recommandée** : **Monolithe modulaire MVP**. Microservices prématurés pour volume B2B. Refactorisation possible ultérieurement si scaling nécessaire.
3. **Gestion fichiers attachés messages (images, PDF) ?**
    - **Contexte** : MVP actuel texte uniquement
    - **Options** :
        - Intégration Néon Storage (S3-compatible) avec URLs signées temporaires
        - Stockage base64 inline database (limite 5MB, non recommandé)
        - Service externe Cloudinary (coût additionnel)
    - **Décision recommandée** : **Phase 2 si demande clients**. Complexité upload/virus scanning/quotas non justifiée MVP. Workaround temporaire : clients envoient fichiers email parallèle.

Questions produit :

4. **Tests end-to-end automatisés MVP ou post-MVP ?**

- **Contexte** : tests unitaires Jest prévus, tests E2E Playwright optionnels
- **Trade-off** :
  - Avec E2E : détection régressions automatique, confiance déploiements, temps setup 1 semaine
  - Sans E2E : tests manuels QA suffisants volume limité, gain temps développement
- **Décision recommandée** : **Post-MVP**. Tests manuels checklist suffisants pilote. Automatisation rentable uniquement si déploiements fréquents (> 1/semaine).

5. **Export PDF conversations clients ?**

   - **Contexte** : clients B2B peuvent vouloir archiver échanges
   - **Options** :
     - Export HTML via bouton "Télécharger conversation" puis impression navigateur
     - Génération PDF backend via bibliothèque (puppeteer overhead)
     - Export JSON brut développeurs uniquement
   - **Décision recommandée** : **Phase 2**. Export HTML MVP suffit. PDF génération backend si demande forte post-pilote (effort 2-3 jours développement).
6. **Notifications push navigateur activées par défaut ?**

   - **Contexte** : notifications desktop via Browser Notification API disponibles
   - **Trade-off** :
     - Activées : utilisateurs alertés immédiatement nouveaux messages hors onglet
     - Désactivées par défaut : moins intrusif, nécessite opt-in explicite
   - **Décision recommandée** : **Opt-in avec prompt première connexion**. RGPD favorise consentement explicite. Taux acceptation attendu 40-60%.

Questions infrastructure :

7. **Stratégie backup base données ?**

   - **Contexte** : Néon backups automatiques quotidiens inclus plan gratuit
   - **Question** : backups manuels additionnels nécessaires avant migrations critiques ?
   - **Décision recommandée** : **Oui, backup manuel via pg_dump avant chaque migration production**. Sécurité additionnelle restauration rapide si migration échoue.
8. **Environnement staging réplique exacte production ?**

   - **Contexte** : actuellement staging prévu base données séparée
   - **Question** : données anonymisées production copiées staging pour tests réalistes ?
   - **Décision recommandée** : **Non MVP, oui post-pilote**. Complexité anonymisation RGPD non justifiée. Données fixtures suffisantes tests initiaux. Script anonymisation phase 2 si debugging production requis.

9. **CDN frontend nécessaire performances ?**

    - **Contexte** : Vercel inclut CDN global automatiquement
    - **Question** : configuration CDN custom (Cloudflare) apporte avantages additionnels ?

Décision recommandée : **Non, Vercel CDN suffisant**. Edge network Vercel couvre 70+ régions avec latence < 50ms Europe. Cloudflare ajouterait complexité sans gain mesurable volume B2B localisé France/Europe.

10. **Monitoring coûts infrastructure automatisé ?**
    - **Contexte** : risque dépassement quotas gratuits si usage inattendu
    - **Question** : alertes budgétaires configurées services cloud ?
    - **Décision recommandée** : **Oui, alertes 80% quota**. Configuration Railway/Vercel/Néon webhooks notifiant Slack si consommation > 80% limites gratuites. Script hebdomadaire extraction coûts actuels.

Questions métier :

11. **Priorisation conversations en attente ?**

    - **Contexte** : actuellement FIFO (first-in first-out)
    - **Question** : clients premium ou urgences doivent-ils être priorisés ?
    - **Options** :
        - FIFO strict : équité totale, simplicité implémentation
        - Score priorité : (urgence + tier client + temps attente) complexité métier
        - Tags urgence manuels client : risque abus
    - **Décision recommandée** : **FIFO MVP avec tag urgence optionnel client**. Ajout score priorité phase 2 si conseillers remontent besoin. Effort développement 3-5 jours.

12. **Historique conversations accessibles combien temps ?**

    - **Contexte** : obligation légale conservation 3 ans, mais accès interface client ?
    - **Question** : client voit conversations des 6 derniers mois ou totalité ?
    - **Décision recommandée** : **6 derniers mois interface, totalité sur demande support**. Pagination performances si milliers conversations anciennes. Export complet via RGPD demande accès données.

13. **Gestion multi-entreprises pour groupes B2B ?**

    - **Contexte** : certains clients B2B gèrent plusieurs filiales
    - **Question** : utilisateur unique peut-il basculer entre plusieurs comptes entreprise ?
    - **Décision recommandée** : **Phase 2 si demande**. MVP 1 utilisateur = 1 entreprise. Refactorisation schema nécessaire (table user_companies jointure). Effort 1 semaine développement.

Questions équipe :

14. **Formation conseillers prévue combien temps avant lancement ?**

- **Contexte** : adoption réussie nécessite maîtrise outil
- **Question** : 2 heures formation suffisantes ou journée complète ?
- **Décision recommandée** : **Session 2h initiale + 2h pratique guidée**. Formation théorique fonctionnalités puis simulation conversations réelles environnement staging. Support documentation vidéo screencasts.

15. **Qui gère modération/escalade conversations problématiques ?**

    - **Contexte** : clients mécontents, insultes potentielles
    - **Question** : procédure escalation définie ? Rôle modérateur dédié ?
    - **Décision recommandée** : **Escalation manager support existant**. Bouton "Signaler conversation" conseillers envoie email manager avec transcript. Pas de rôle modérateur dédié volume B2B limité. Bannissement client rare nécessite validation direction.

Questions légales :

16. **Consentement explicite tracking conversations RGPD ?**

    - **Contexte** : conversations stockées 3 ans, cookies analytics
    - **Question** : banner consentement cookies + mention archivage conversations requis ?
    - **Décision recommandée** : **Oui, banner conformité cookie + mention CGU**. Texte type : "En utilisant ce chat, vous acceptez la conservation de vos échanges pendant 3 ans à des fins de support et obligations légales (article X CGU)". Validation juridique pré-lancement.

17. **Assurance cyber-risques couvre fuite données conversations ?**

    - **Contexte** : risque piratage base données exposant échanges clients
    - **Question** : police assurance entreprise couvre cyber-incidents ?
    - **Décision recommandée** : **Vérifier couverture existante avec assureur**. Montant garanti minimum 500k€ recommandé. Si non couvert, souscrire assurance cyber dédiée (coût 1-3k€/an PME).

Questions techniques avancées :

18. **Mécanisme retry automatique messages échoués côté client ?**

    - **Contexte** : perte connexion réseau pendant envoi message
    - **Question** : nombre tentatives retry et backoff exponentiel ?
    - **Décision recommandée** : **3 tentatives avec backoff 1s/3s/10s**. Au-delà, message marqué "échec envoi" avec bouton "Renvoyer" manuel. Évite boucles infinies si serveur down prolongé.

19. **Rate limiting Socket.IO par utilisateur ou par IP ?**

    - **Contexte** : protection DoS événements WebSocket
    - **Question** : NAT entreprises B2B partagent IP, limiter par userId ?

- **Décision recommandée** : **Rate limit par userId authentifié**. IP partagées fausseraient limites. Redis stockage compteurs si multi-serveurs, sinon Map mémoire suffisante.

20. **Migration données conversations depuis ancien système ?**

    - **Contexte** : historique emails support existant
    - **Question** : import ancien historique dans nouveau chat ou séparation ?
    - **Décision recommandée** : **Séparation MVP, lien vers ancien système**. Import complexité migration données non structurées (emails) vers schema conversations. Lien interface "Voir historique emails précédent" vers ancien outil. Migration phase 2 si demande forte.

Synthèse décisions critiques nécessitant validation PO :
Copier le tableau

Question	Recommandation	Impact planning	Validation requise avant
Redis MVP	Non, différer phase 2	-3 jours dev	Sprint planning
Tests E2E automatisés	Non MVP	-5 jours dev	Sprint planning
Export PDF conversations	Phase 2	-2 jours dev	Sprint planning
Priorisation conversations	FIFO MVP	-4 jours dev si score priorité	Semaine 2
Formation conseillers	4h total (2h théorie + 2h pratique)	Budget formation	Semaine 5
Assurance cyber-risques	Vérification couverture	Budget assurance	Avant lancement pilote

Actions immédiates :

1. **Semaine prochaine :** Réunion validation PO/Tech Lead décisions questions 1-6 (architecture/produit)
2. **Semaine 2 :** Validation juridique DPO questions 16-17 (RGPD/assurance)
3. **Semaine 4 :** Préparation formation conseillers questions 14-15 (équipe)

16. Annexes

Exemple payload complet conversation :

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "clientId": "123e4567-e89b-12d3-a456-426614174000",
 "advisorId": "789e0123-e45b-67c8-d901-234567890abc",
```

```
"status": "ACTIVE",
"subject": "Retard livraison commande #CMD-2024-0156",
"metadata": {
  "relatedOrderId": "ord_abc123",
  "clientCompany": "Entreprise ABC SAS",
  "clientSegment": "premium",
  "tags": ["livraison", "urgence"],
  "transferHistory": [
    {
      "fromAdvisorId": "previous-advisor-id",
      "toAdvisorId": "789e0123-e45b-67c8-d901-234567890abc",
      "transferredAt": "2024-01-15T10:25:00Z",
      "reason": "Transfert expertise logistique"
    }
  ]
},
"createdAt": "2024-01-15T10:20:00Z",
"claimedAt": "2024-01-15T10:21:30Z",
"closedAt": null,
"resolutionCategory": null,
"resolutionNote": null,
"rating": null,
"client": {
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "name": "Jean Dupont",
  "email": "jean.dupont@entreprise-abc.fr",
  "role": "CLIENT",
  "company": {
    "id": "comp-xyz-789",
    "name": "Entreprise ABC SAS",
    "siret": "12345678901234"
  }
},
```

```
"advisor": {
  "id": "789e0123-e45b-67c8-d901-234567890abc",
  "name": "Marie Martin",
  "email": "marie.martin@support.com",
  "role": "ADVISOR"
},
"messages": [
  {
    "id": "msg-001",
    "conversationId": "550e8400-e29b-41d4-a716-446655440000",
    "senderId": "123e4567-e89b-12d3-a456-426614174000",
    "senderName": "Jean Dupont",
    "senderRole": "CLIENT",
    "content": "Bonjour, ma commande devait arriver hier mais toujours rien
reçu. Pouvez-vous vérifier ?",
    "type": "TEXT",
    "sentAt": "2024-01-15T10:20:00Z",
    "readAt": "2024-01-15T10:21:35Z"
  },
  {
    "id": "msg-002",
    "conversationId": "550e8400-e29b-41d4-a716-446655440000",
    "senderId": "system",
    "senderName": "Système",
    "senderRole": "SYSTEM",
    "content": "Marie Martin a pris en charge votre demande",
    "type": "SYSTEM",
    "sentAt": "2024-01-15T10:21:30Z",
    "readAt": null
  },
  {
    "id": "msg-003",
    "conversationId": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "senderId": "789e0123-e45b-67c8-d901-234567890abc",
    "senderName": "Marie Martin",
    "senderRole": "ADVISOR",
    "content": "Bonjour Jean, je vérifie immédiatement le statut de votre
commande #CMD-2024-0156...",
    "type": "TEXT",
    "sentAt": "2024-01-15T10:22:00Z",
    "readAt": "2024-01-15T10:22:05Z"
  }
 ],
 "unreadCount": 0
}
```

Variables environnement complètes :

```
# Backend .env
NODE_ENV=production
PORT=3001

# Néon
DATABASE_URL=postgresql://postgres:password@db.xxx.Néon.co:5432/postgres
Néon_URL=https://xxx.Néon.co
Néon_ANON_KEY=eyJxxx...
Néon_SERVICE_KEY=eyJxxx...

# JWT
JWT_SECRET=super-secret-change-in-production-min-32-chars
JWT_EXPIRES_IN=1h
REFRESH_TOKEN_SECRET=another-super-secret-min-32-chars
REFRESH_TOKEN_EXPIRES_IN=30d

# Redis (optionnel phase 2)
```

```
REDIS_URL=redis://default:password@redis.upstash.io:6379

# CORS
ALLOWED_ORIGINS=https://app.example.com,https://admin.example.com

# Rate limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100

# Sentry
SENTRY_DSN=https://xxx@sentry.io/xxx
SENTRY_ENVIRONMENT=production

# Email notifications (optionnel)
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASSWORD=SG.xxx
EMAIL_FROM=support@example.com

# Frontend .env
VITE_API_URL=https://api.example.com
VITE_WS_URL=wss://api.example.com
VITE_Néon_URL=https://xxx.Néon.co
VITE_Néon_ANON_KEY=eyJxxx...
VITE_SENTRY_DSN=https://xxx@sentry.io/xxx
VITE_ENVIRONMENT=production
```

Exemple queries Prisma optimisées :

```
// Récupération conversations conseiller avec pagination
async function getAdvisorConversations(advisorId, page = 1, perPage = 20) {
```

```
const skip = (page - 1) * perPage;
 const [conversations, total] = await Promise.all([
  prisma.conversation.findMany({
    where: {
      advisorId,
      status: { in: ['ACTIVE', 'WAITING'] }
    },
    include: {
      client: {
        select: {
          id: true,
          name: true,
          email: true,
          company: {
            select: { name: true }
          }
        }
      },
      messages: {
        take: 1,
        orderBy: { sentAt: 'desc' },
        select: {
          content: true,
          sentAt: true
        }
      },
      _count: {
        select: {
          messages: {
            where: {
              senderId: { not: advisorId },
              readAt: null
            }
```

```
          }
        }
      }
    },
    orderBy: { createdAt: 'desc' },
    skip,
    take: perPage
  }),

  prisma.conversation.count({
    where: {
      advisorId,
      status: { in: ['ACTIVE', 'WAITING'] }
    }
  })
]);
 return {
  data: conversations.map(conv => ({
    ...conv,
    lastMessage: conv.messages[0],
    unreadCount: conv._count.messages
  })),
  pagination: {
    page,
    perPage,
    total,
    totalPages: Math.ceil(total / perPage)
  }
};
}

// Création conversation avec transaction atomique
async function createConversation(clientId, subject, initialMessage) {
```

```
return await prisma.$transaction(async (tx) => {
  // Vérification limite 3 conversations actives
  const activeCount = await tx.conversation.count({
    where: {
      clientId,
      status: { in: ['ACTIVE', 'WAITING'] }
    }
  });

  if (activeCount >= 3) {
    throw new Error('Maximum 3 active conversations per client');
  }

  // Création conversation
  const conversation = await tx.conversation.create({
    data: {
      clientId,
      subject,
      status: 'WAITING',
      metadata: {}
    }
  });

  // Création message initial
  await tx.message.create({
    data: {
      conversationId: conversation.id,
      senderId: clientId,
      content: initialMessage,
      type: 'TEXT'
    }
  });
```

```
  return conversation;
 });
}
```

Scripts utilitaires :

// scripts/seedDatabase.js - Génération données test

```
const { PrismaClient } = require('@prisma/client');
const { faker } = require('@faker-js/faker');
const bcrypt = require('bcrypt');

const prisma = new PrismaClient();

async function seed() {
 console.log('Seeding database...');
  // Création entreprises
 const companies = await Promise.all(
   Array.from({ length: 10 }, async () =>
     prisma.company.create({
       data: {
         name: faker.company.name(),
         siret: faker.string.numeric(14)
       }
     })
   )
 );
  // Création clients
 const clients = await Promise.all(
   companies.map(company =>
     prisma.user.create({
       data: {
         email: faker.internet.email(),
         name: faker.person.fullName(),
```

```
        password: await bcrypt.hash('password123', 10),
        role: 'CLIENT',
        companyId: company.id
      }
    })
  )
);
 // Création conseillers
const advisors = await Promise.all(
  Array.from({ length: 5 }, async () =>
    prisma.user.create({
      data: {
        email: faker.internet.email(),
        name: faker.person.fullName(),
        password: await bcrypt.hash('password123', 10),
        role: 'ADVISOR'
      }
    })
  )
);
 // Création conversations avec messages
for (const client of clients.slice(0, 20)) {
  const advisor = faker.helpers.arrayElement(advisors);

  const conversation = await prisma.conversation.create({
    data: {
      clientId: client.id,
      advisorId: advisor.id,
      subject: faker.lorem.sentence(),
      status: faker.helpers.arrayElement(['WAITING', 'ACTIVE', 'RESOLVED']),
      metadata: {}
    }
  });
```

```
  // Messages aléatoires
  const messageCount = faker.number.int({ min: 3, max: 15 });
  for (let i = 0; i < messageCount; i++) {
    await prisma.message.create({
      data: {
        conversationId: conversation.id,
        senderId: i % 2 === 0 ? client.id : advisor.id,
        content: faker.lorem.paragraph(),
        type: 'TEXT',
        sentAt: faker.date.recent({ days: 7 })
      }
    });
  }
}
 console.log(`Seeded ${companies.length} companies, ${clients.length}
clients, ${advisors.length} advisors`);
}

seed()
 .catch(console.error)
 .finally(() => prisma.$disconnect());

# scripts/backup.sh - Backup automatisé base données
#!/bin/bash

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="./backups"
BACKUP_FILE="$BACKUP_DIR/backup_$DATE.sql"

mkdir -p $BACKUP_DIR

echo "Starting backup at $DATE..."
```

```
pg_dump $DATABASE_URL > $BACKUP_FILE

if [ $? -eq 0 ]; then
 echo "Backup successful: $BACKUP_FILE"
 gzip $BACKUP_FILE
  # Nettoyage backups > 30 jours
 find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -delete
 echo "Cleanup old backups completed"
else
 echo "Backup failed!"
 exit 1
fi
```

Checklist lancement production :
Semaine -1 (avant lancement pilote) :

- Tous tests automatisés passent (unit + integration)
- Tests manuels QA checklist complétée
- Audit sécurité externe commandé et validé
- Performance tests k6 exécutés (200 users simultanés OK)
- Backup base données production effectué
- Certificats SSL valides et renouvelables automatiquement
- Variables environnement production vérifiées
- Monitoring Sentry/Uptime configuré et testé
- Documentation utilisateur finalisée
- Formation conseillers planifiée et confirmée

Jour J (lancement pilote) :

- Déploiement backend production exécuté
- Déploiement frontend production exécuté
- Migrations base données appliquées avec succès
- Health checks tous services retournent 200 OK
- Test end-to-end conversation complète production
- Email annonce envoyé 50 clients pilote
- Message banner interface activé : "Nouveau : chattez en direct avec nos conseillers"
- Conseillers connectés et prêts

Semaine +1 (post-lancement) :

- Revue métriques quotidiennes (adoption, satisfaction, bugs)

- Session feedback conseillers organisée
- Correctifs bugs critiques déployés si nécessaires
- Communication clients pilote : sondage satisfaction
- Documentation mise à jour basée retours terrain

Glossaire technique :

- **B2B** : Business-to-Business, commerce entre entreprises
- **JWT** : JSON Web Token, standard authentification stateless
- **MVP** : Minimum Viable Product, version minimale fonctionnelle
- **P95** : 95e percentile, métrique performance (95% requêtes sous valeur)
- **RLS** : Row Level Security, sécurité niveau ligne PostgreSQL
- **SLA** : Service Level Agreement, engagement niveau service
- **SSR** : Server-Side Rendering, rendu côté serveur
- **TTL** : Time To Live, durée vie cache
- **WebSocket** : Protocole communication bidirectionnel temps réel
- **XSS** : Cross-Site Scripting, vulnérabilité injection code malveillant

Ressources externes :

- Documentation Socket.IO : https://socket.io/docs/v4/
- Documentation Prisma : https://www.prisma.io/docs
- Documentation Néon : https://neon.com/docs
- Guide sécurité OWASP : https://owasp.org/www-project-web-security-testing-guide/
- Règlement RGPD : https://www.cnil.fr/fr/reglement-europeen-protection-donnees